FACULTAD DE INFORMÁTICA
MÁSTER DE INGENIERÍA DEL SOFTWARE



SolidarianID

Documentación de las pruebas realizadas

Grupo 3:
Antonio Pérez Serrano
Fabián Sola Durán
Manuel Bernabé Rodríguez
Pablo Peris Solano
Sergio Ros Liarte

Control de la Calidad y Pruebas del Software
12/1/2026

Índice

1. Introducción **4**

2. Plan de pruebas **4**
2.1. Introducción 4
2.2. Recursos Humanos 4
2.3. Alcance 5
2.4. Fuera del Alcance 6
2.5. Nueva funcionalidad 7
2.5.1. Revisión del código (*ESLint*) 8
2.5.2. Análisis estático completo (SonarQube) 8
2.5.3. Pruebas unitarias (Jest) y evaluación por mutación (Stryker) . . . 8
2.5.4. Pruebas frontend (jest-dom) en la aplicación de administración . . 9
2.6. Pruebas de integración 9
2.6.1. Pruebas backend de los repositorios 9
2.6.2. Pruebas backend de endpoints (.rest, supertest y base de datos efímera con Testcontainers) 10
2.6.3. Pruebas end-to-end (Cypress) 11
2.6.4. Pruebas metamórficas (unitaria, API Rest y/o front-end) y estrategia 11
2.6.5. Pruebas de regresión (selección, minimización o priorización) y casos derivados 11
2.6.6. Pruebas de Rendimiento y Carga 11
2.6.7. Pruebas de Aceptación de usuario 11
2.7. Consideraciones de Infraestructura 12
2.8. Suposiciones 12
2.9. Riesgos 12
2.10. Criterio de finalización 13

3. Resolución de las pruebas **14**
3.1. Pruebas Unitarias 14
3.1.1. Pruebas de Caja Blanca 14
3.1.2. Pruebas de Caja Negra 19
3.2. Pruebas de Análisis Estático 22
3.3. Pruebas de Mutación 26
3.4. Pruebas de Front-End 29
3.5. Pruebas de Back-End 30
3.6. Pruebas de End-To-End 33
3.6.1. Indicaciones para ejecutar estas pruebas (para la corrección) . . . 33
3.6.2. Reutilización y mantenibilidad 33
3.6.3. Escenarios E2E implementados 33
3.6.4. Preparación de estado y estabilidad de las pruebas 34
3.6.5. Ejecución y evidencias 34
3.7. Pruebas de Rendimiento 35
3.7.1. Entorno de pruebas y realismo del escenario 35
3.7.2. Herramienta utilizada y diseño del plan de pruebas 35
3.7.3. Ejecución de las pruebas 36

3.7.4. Generación del reporte de resultados 36
3.7.5. Resultados obtenidos 36
3.7.6. Análisis e interpretación de los resultados 37
3.8. Pruebas Metamórficas 38
3.8.1. Pruebas metamórficas unitarias 38
3.8.2. Pruebas metamórficas e2e 40
3.8.3. Pruebas metamórficas con Katalon 41
3.9. Pruebas de Regresión 44

4. Conclusión **45**

Índice de figuras

1. Resumen del reporte de SonarQube para *SolidarianID*: estado del análisis, *Quality Gate* y métricas principales. 25
2. Resultados de testing de mutaciones en usuarios 28
3. Resultados de testing de mutaciones en Value Objects 28
4. Evolución de los tiempos de respuesta durante la prueba 37
5. Pruebas metamórficas unitarias 39
6. Pruebas metamórficas del login con Katalon 42

1. Introducción

Este documento recoge el plan de pruebas y la ejecución de las distintas pruebas realizadas sobre el sistema desarrollado, con el objetivo de comprobar su correcto funcionamiento y mejorar su calidad. Las pruebas nos han servido para detectar errores de forma temprana y aportar mayor confianza en la estabilidad del software.

A lo largo del trabajo se describen las diferentes estrategias de prueba aplicadas, desde pruebas unitarias y análisis estático hasta pruebas de integración, end-to-end, rendimiento y regresión. En primer lugar, se presenta el plan de pruebas, que define el alcance, los recursos, los riesgos y los criterios de finalización. Posteriormente, se detalla la resolución de las pruebas ejecutadas, mostrando cómo se han validado los distintos componentes del sistema.

En conjunto, este trabajo ofrece una visión práctica del proceso de pruebas aplicado al proyecto, justificando las técnicas y herramientas utilizadas para asegurar un software robusto y mantenible.

También se adjunta al equipo docente el enlace a nuestro Repositorio de GitHub [1] al que previamente han sido invitados ambos docentes. Se recomienda encarecidamente revisar el respectivo directorio dedicado a esta asignatura llamado `ccps`.

2. Plan de pruebas

2.1. Introducción

Este plan define la estrategia y el proceso de pruebas para el MVP implementado de SolidarianID por parte del grupo 3, centrado en la aplicación de administración (monolito NestJS) y en las APIs de los microservicios de la aplicación principal, concretamente Administración y el CRUD de usuarios, comunidades, causas, acciones y eventos. El objetivo es asegurar que la aplicación bajo prueba (AUT) se comporta conforme a los requisitos del MVP, que los defectos encontrados se corrigen antes de la entrega o quedan planificados y justificados, y que el producto mantiene un nivel de calidad verificable mediante revisión de código, análisis estático, pruebas automáticas y pruebas de rendimiento. Este plan no entra en detallar la metodología porque se asume un plan de pruebas ágil, y el detalle se desarrolla a lo largo de los siguientes apartados.

2.2. Recursos Humanos

Los recursos humanos de este plan de pruebas se pueden observar en la Tabla 1.

Nombre del tester	Participación	Rol	Tarea asignada
Antonio Pérez	30 %	Desarrollador & Agile tester	Pruebas unitarias en módulos críticos, prueba de mutación con Stryker y primera capa de regresión automática en CI.
Pablo Peris	25 %	Desarrollador & tester	Pruebas de endpoints con ficheros `.rest` y asertos con supertest, apoyo en preparación de datos semilla y BBDD de pruebas.
Manuel Bernabé	15 %	Product Owner	Validación de criterios de aceptación y ejecución de UAT sobre flujos de administración priorizados, registro de feedback e incidencias.
Fabián Sola	15 %	Arquitecto de software	Asegurar infraestructura de pruebas, entornos y dependencias (BBDD, Redis, mensajería), revisión de resultados SonarQube y orientación de correcciones estructurales.
Sergio Ros Liarte	15 %	Agile Tester	Coordinación de E2E con Cypress, revisión de evidencias, apoyo en priorización y reproducción de defectos, y soporte en pruebas de rendimiento con JMeter.

Cuadro 1: Asignación de roles, participación y tareas para la ejecución del plan de pruebas

2.3. Alcance

El alcance de este plan cubre tanto actividades de control de calidad del código como pruebas sobre la aplicación bajo prueba.

En primer lugar, se realizará la revisión del código con `ESLint` mediante el formato *Flat Config*, integrando `Prettier` para la guía de estilo y aplicando las reglas recomendadas de TypeScript con comprobación de tipos. Se han establecido configuraciones específicas para entornos de pruebas (Jest y Cypress), justificando decisiones relevantes como la flexibilización de reglas estrictas (por ejemplo, el uso de any explícito o la relajación de comprobaciones de seguridad en los tests) para agilizar el desarrollo sin comprometer la calidad, dejando trazabilidad en el repositorio mediante el archivo eslint.config.mjs.

En segundo lugar, se ejecutará un análisis estático completo con SonarQube, interpretando sus métricas (code smells, vulnerabilidades, duplicidad y cobertura) y verificando el cumplimiento del quality gate acordado. Además, cuando aparezcan hallazgos, se documentará su causa y la acción correctiva o la aceptación del riesgo si procede.

En tercer lugar, se implementarán pruebas unitarias con Jest en algunos módulos del proyecto siguiendo técnicas de caja blanca (caminos básicos, condiciones y bucles) y caja negra (particiones de equivalencia y casos límite) y después se evaluará la calidad de

dichos casos de prueba mediante pruebas de mutación con Stryker, analizando mutantes vivos y actuando al menos sobre uno de ellos para ilustrar la técnica, ya sea marcándolo como equivalente o añadiendo casos que lo eliminen.

En cuarto lugar, se definirán pruebas de frontend con jest-dom para la aplicación de administración, duplicando el DOM en los casos de prueba y validando renderizado, estados, validaciones y accesibilidad básica de formularios y tablas (sin extender este alcance al frontend público de la aplicación principal).

En quinto lugar, se definirán pruebas backend de endpoints considerando que el proyecto contiene la carpeta `ccps/requests` con ficheros `.rest` para pruebas manuales y reproducibles, que el comportamiento de endpoints se comprobará además con asertos automatizados usando supertest, y que el acceso a una base de datos de pruebas se configurará con TestContainers (para la mayoría de casos) y variables de entorno (para el caso de los test E2E con Cypress). Todo ello con el objetivo de no usar nunca la base de datos de producción.

En sexto lugar, se definirá al menos una prueba end-to-end con Cypress que cubra un caso de uso del MVP.

En séptimo lugar, se define un plan de pruebas de rendimiento con JMeter para el caso de uso del MVP que más sobrecarga genera en nuestra aplicación (solicitud de creación de comunidades) usando listeners, aserciones y controladores, y se comprobarán límites establecidos para métricas como APDEX .

En octavo lugar, se incluye un plan de pruebas metamórficas a nivel unitario y/o API REST y/o E2E, adoptando una estrategia basada en relaciones metamórficas que permitan validar propiedades incluso cuando el oráculo exacto es costoso.

En noveno y último lugar, se define una estrategia de pruebas de regresión basada en *retest-all* de forma que tras cambios se ejecuten todos los test en un proceso de integración continua. Nos habría gustado averiguar la configuración de Jest para aplicar una estrategia de pruebas de regresión selectivas en nuestro proceso de integración continua. Lamentablemente, no hemos podido indagar en el siguiente enlace [2] e idear una manera de abordar esta estrategia.

Este alcance se aplica a las funcionalidades del MVP que se han implementado: Administración y CRUD de usuarios, comunidades, causas, acciones y eventos, incluyendo validaciones de datos, permisos por rol y consistencia de estados en los procesos de creación y edición.

2.4. Fuera del Alcance

Quedan fuera del alcance de esta versión las funcionalidades no implementadas en el MVP o no incluidas por decisión del equipo. En concreto, no se prueba el frontend público de la aplicación principal (interfaz para usuarios registrados/no registrados fuera de la app de administración), ni se cubren funciones de red social (seguimiento, mensajería, compartición) ni flujos de donaciones o integración de pagos, y tampoco se realizan pruebas

exhaustivas de integraciones externas no presentes. Asimismo, no se realiza un pentest completo ni auditorías legales detalladas, aunque sí se contemplan comprobaciones básicas de seguridad (autorización, validación de entradas, y separación de entornos) como parte de las pruebas técnicas. (TsArch, Cucumber, TestContainer, Regresión selectiva en caso de que quede fuera).

Aunque el uso de TestContainers se consideraba fuera del alcance hemos conseguido de manera autodidacta su implementación. Es un punto adicional en nuestra entrega. Lamentablemente, Cucumber y el uso de TsArch han quedado fuera de nuestro alcance por falta de tiempo; lo cierto es que TsArch nos habría venido fenomenal para mantener una nomenclatura y estructura del código homogénea entre los distintos integrantes del grupo. Y con respecto a Cumbumber, parte de los test de aceptación ya fueron definidos explícitamente para la asignatura de Gestión Ágiles de Proyectos por lo que hubiera sido otro recurso valioso.

2.5. Nueva funcionalidad

La nueva funcionalidad a validar se centra en la aplicación de administración y en las APIs que soportan las operaciones de gestión. Se verificará parte del **CRUD de usuarios** (alta, consulta, edición y desactivación o borrado según el diseño), comprobando **validaciones**, **unicidad** y **mensajes de error consistentes**, y asegurando que la **autorización** impide operaciones a roles no permitidos. Asimismo, se validará el **CRUD de comunidades**, incluyendo creación, edición, cambios de estado y relaciones asociadas, comprobando que los cambios se reflejan en listados y detalles y que no se permiten **estados incoherentes**. Se validará el **CRUD de causas**, garantizando que se exigen los campos obligatorios, que las asociaciones relevantes (por ejemplo con ODS si aplica en el MVP) se conservan y se consultan correctamente, y que los estados de revisión o aprobación (si existen) siguen el flujo esperado. Del mismo modo, se validará el **CRUD de acciones** y su relación con causas, incluyendo la gestión de objetivos cuantificables cuando aplique, y el **CRUD de eventos**, verificando fecha/hora, lugar y la relación con la acción o causa correspondiente. Además se realizarán endpoints de análisis de datos para poder facilitar a la vista del administrador por parte del proyecto de comunidades.

Para esta nueva funcionalidad se aplicarán distintos tipos de prueba, cubriendo la lógica de validación y consistencia, la interacción con la persistencia, el correcto comportamiento de los endpoints y la experiencia de administración en los flujos principales. Se incluirá al menos un recorrido completo de extremo a extremo que represente un caso de uso típico: autenticación en la administración, creación de una entidad principal (p. ej., comunidad o causa), modificación de un atributo y verificación posterior en listado y detalle, dejando evidencias de la ejecución.

Respecto a **pruebas metamórficas**, se adoptará una estrategia basada en relaciones que deben mantenerse entre ejecuciones relacionadas. Por ejemplo, en listados con paginación y filtros, la unión de páginas consecutivas debe equivaler al conjunto completo ordenado bajo el mismo criterio, y un filtro más restrictivo debe devolver un resultado que sea subconjunto del listado sin filtrar. En operaciones CRUD, tras una creación exitosa, una lectura posterior debe mantener los invariantes del recurso (existencia y campos fijados), y

una actualización sin cambios debe ser idempotente (no altera el estado observable). Estas relaciones permiten detectar defectos sin depender de un oráculo exacto del resultado final, lo cual es útil cuando los datos de prueba pueden variar.

Para **regresión**, se definirá una suite mínima crítica priorizada por riesgo que incluya autenticación en administración, lectura de listados principales y una operación CRUD representativa por entidad (usuarios, comunidades, causas, acciones y eventos). Tras cada cambio se ejecutará primero esta suite y, si el cambio afecta a una entidad concreta, se ampliará con los casos relacionados (selección por impacto), manteniendo la minimización para evitar redundancias y favorecer tiempos de ejecución razonables.

2.5.1. Revisión del código (*ESLint*)

En esta fase se realizará una revisión del código apoyada en un **control estático con *ESLint***, integrando *Prettier* para la guía de estilo y las reglas recomendadas de *TypeScript* con comprobación de tipos, configuradas específicamente para el proyecto SolidarianID.

La configuración utiliza el formato moderno *Flat Config* (`eslint.config.mjs`) y define perfiles específicos para el código fuente y los entornos de pruebas (*Jest* y *Cypress*). Se justifican decisiones relevantes, como la desactivación de reglas conflictivas con el formateador o la flexibilización de ciertas restricciones de tipado (como `no-explicit-any` o comprobaciones estrictas en los tests) para agilizar el desarrollo. El objetivo es reducir defectos de calidad antes de ejecutar el resto de pruebas, unificando estilos y detectando patrones inseguros, dejando evidencias de ejecución en el repositorio.

2.5.2. Análisis estático completo (SonarQube)

Por otro lado, Se ejecutará un **análisis estático del código con SonarQube** integrado en nuestro pipeline de integración continua y se interpretarán sus resultados en términos de mantenibilidad, fiabilidad y seguridad. Se revisarán indicadores como *bugs*, *vulnerabilities*, *code smells*, duplicidad y cobertura. Los hallazgos se documentarán y adjuntarán.

2.5.3. Pruebas unitarias (Jest) y evaluación por mutación (Stryker)

Se implementarán **pruebas unitarias con Jest en módulos del MVP**, priorizando lógica de validación, permisos, mapeos DTO→dominio y reglas básicas asociadas a los CRUD de usuarios, comunidades, causas, acciones y eventos. Para el diseño de casos de prueba se aplicarán técnicas de caja blanca y de caja negra de forma complementaria, procurando cubrir tanto la estructura interna del código como el comportamiento esperado desde el punto de vista funcional.

Con respecto a las **técnicas de pruebas de caja blanca** se aplicarán, por un lado, pruebas de complejidad ciclomática o caminos básicos para asegurar que las rutas principales de decisión quedan cubiertas cuando existan ramificaciones relevantes, por otro lado pruebas de condición para verificar expresiones lógicas (incluyendo combinaciones

típicas como verdadero/falso y casos donde una condición domina a otra), y finalmente pruebas de bucles para cubrir escenarios representativos en iteraciones, como colecciones vacías, un único elemento y varios elementos, evitando errores frecuentes de límites en iteración.

Por otro lado, en cuanto a las **técnicas de pruebas de caja negras** se aplicarán pruebas de límites para validar valores frontera y extremos de campos y entradas (por ejemplo tamaños mínimos y máximos, rangos numéricos, fechas y longitudes), y pruebas de partición de equivalencia para agrupar entradas en clases válidas e inválidas reduciendo redundancia, de forma que con pocos casos se obtenga buena cobertura funcional. Con todo ello, se busca que las pruebas no se limiten a "que pase", si no a detectar fallos reales en validaciones, reglas y transformaciones.

Una vez que se tengan las pruebas unitarias implementadas, se realizará una **evaluación de la calidad de los casos mediante una prueba de mutación con Stryker**, con el objetivo de medir la capacidad de los tests para detectar modificaciones artificiales en el código. Si la prueba de mutación deja mutantes vivos, se interpretarán las causas de esta supervivencia, diferenciando casos de mutante equivalente frente a casos de insuficiente calidad de los tests. Se actuará al menos sobre un mutante vivo para ilustrar la técnica, ya sea marcándolo como equivalente con una justificación clara o definiendo nuevos casos de prueba que lo eliminen, dejando trazabilidad del cambio y comparando el resultado antes y después.

2.5.4. Pruebas frontend (jest-dom) en la aplicación de administración

Se definirán **pruebas frontend** de la aplicación de administración utilizando **jest-dom**, permitiendo duplicar la estructura del DOM en los casos de prueba y validar el comportamiento de componentes relevantes del MVP. Se comprobará renderizado correcto, presencia de elementos críticos (formularios, tablas, botones de acciones), validaciones de campos y mensajes de error, y estados básicos (cargando, vacío, error, éxito) en operaciones típicas de gestión. Estas pruebas se limitan a la aplicación de administración, quedando explícitamente fuera del alcance el front público de la aplicación principal.

2.6. Pruebas de integración

2.6.1. Pruebas backend de los repositorios

Para garantizar la correcta validación de los repositorios usados para comunicarse con la base de datos se definirán pruebas de integración de los repositorios con sus respectivas bases de datos utilizando *supertest* y bases de datos de pruebas.

2.6.2. Pruebas backend de endpoints (.rest, supertest y base de datos efímera con Testcontainers)

Para garantizar la correcta validación del backend del MVP, se definen **pruebas de endpoints** que cubren tanto el comportamiento funcional como los aspectos de seguridad y persistencia de datos. Estas pruebas se apoyan en tres pilares principales.

En primer lugar, el proyecto incluye una carpeta `ccps/request` que contiene algunos ficheros `.rest`. Estos ficheros documentan de forma reproducible las peticiones HTTP y las respuestas esperadas de los distintos endpoints relacionados con las operaciones más ejecutadas durante nuestro testing. Esto nos ha facilitado tanto la validación manual como la comprensión del sistema.

En segundo lugar, el comportamiento de los endpoints se valida de manera automatizada mediante `supertest`. A través de aserciones, se comprueba la corrección de los códigos de estado HTTP, la estructura y contenido de los payloads, las validaciones de entrada y la consistencia de las respuestas de error. Se priorizan especialmente las pruebas negativas, como accesos no autorizados por rol, errores de validación y escenarios límite en operaciones CRUD, ya que estos aspectos son críticos en los flujos de administración y control del sistema.

IMPORTANTE

¡Para poder realizar las pruebas e2e recuerda tener activo el *docker daemon* para poder ejecutar los `TestContainers`!

Finalmente, el acceso a la base de datos durante las pruebas se gestiona mediante Testcontainers. Este enfoque permite levantar de forma dinámica contenedores Docker con una base de datos de pruebas, completamente aislada del entorno de producción. De este modo, se evita por diseño cualquier riesgo de ejecutar pruebas contra datos reales y se garantiza que cada ejecución parte de un estado limpio y reproducible.

El uso de Testcontainers aporta varios beneficios clave. En primer lugar, asegura que las pruebas se ejecutan sobre el mismo tipo de infraestructura que se utiliza en producción, reduciendo discrepancias entre entornos. En segundo lugar, simplifica la integración de las pruebas backend en el flujo de integración continua, ya que basta con ejecutar `npm run test:e2e` para levantar el entorno completo, ejecutar las pruebas y destruir los contenedores al finalizar. Aunque el uso de variables de entorno mediante herramientas como `cross-env` es un enfoque válido y útil (tal y como vimos en clase), su gestión se vuelve más compleja en escenarios de CI/CD con múltiples servicios dependientes. Testcontainers abstrae esta complejidad y facilita la ejecución de pruebas end-to-end en plataformas como GitHub Actions, contribuyendo a un flujo de valor altamente automatizado y alineado con una puesta en producción fiable y consistente. De esta forma, puesto que hemos dispuesto de poco tiempo para realizar el proyecto, podemos asegurar que cada endpoint REST expuesto, realiza la funcionalidad esperada.

2.6.3. Pruebas end-to-end (Cypress)

Se definirá **al menos una prueba E2E con Cypress** que cubra un caso de uso real del MVP en la aplicación implementada. El caso mínimo propuesto es: autenticación en la aplicación de administración con credenciales de rol autorizado, edición de un atributo relevante y confirmación final de persistencia. Esta prueba aportará evidencia completa de integración entre UI, API y base de datos de pruebas.

2.6.4. Pruebas metamórficas (unitaria, API Rest y/o front-end) y estrategia

Se definirán **pruebas metamórficas** para el MVP, aplicables a nivel de **unitarias** y/o **API REST**, haciendo uso de **Katalon**, y basadas en **relaciones metamórficas** (p. ej., invariantes e idempotencia) para detectar errores cuando no existe un oráculo exacto o los datos varían; por ejemplo, un **filtro más restrictivo** debe devolver un resultado que sea **subconjunto** del listado sin filtrar (si `GET /causas` devuelve A, entonces `GET /causas?estado=ACTIVA` debe devolver B cumpliendo $B \subseteq A$). Y ver conjuntos disjuntos si forman el total.

2.6.5. Pruebas de regresión (selección, minimización o priorización) y casos derivados

Se definirá una estrategia de **pruebas de regresión test-all** para verificar que los cambios (nuevas funcionalidades o correcciones) no introducen nuevos errores en la funcionalidad existente, además, se ha adoptado este enfoque por su simplicidad y por nuestra falta de tiempo de cara a esta entrega de enero.

2.6.6. Pruebas de Rendimiento y Carga

Las pruebas de rendimiento y carga se ejecutarán con **JMeter** sobre los **endpoints críticos** del *MVP*, siguiendo una secuencia que represente el uso habitual de la administración y las **APIs**. Se realizará primero una prueba base con carga baja, después una prueba de carga con concurrencia moderada, y finalmente una prueba de estrés incrementando usuarios hasta observar degradación; si es necesario, se añadirá una prueba sostenida para comprobar la estabilidad en el tiempo. Se registrarán tiempos de respuesta y tasa de errores, verificando respuestas *HTTP* esperadas, y se evaluará el cumplimiento de umbrales definidos en el plan (incluyendo un objetivo de $\boldsymbol{APDEX} \geq 0{,}8$). Los resultados se revisarán para detectar posibles cuellos de botella y proponer mejoras cuando proceda.

2.6.7. Pruebas de Aceptación de usuario

Las **pruebas de aceptación de usuario (UAT)** se realizarán sobre la **aplicación de administración** para confirmar, desde el punto de vista del **Product Owner**, que los **flujos críticos del MVP** cumplen los **criterios de aceptación**. Manuel Bernabé

ejecutará los recorridos con datos representativos, validando inicio de sesión, navegación y operaciones principales, y registrando incidencias con pasos de reproducción y severidad. La versión se considerará aceptada cuando no existan **defectos bloqueantes** registrados en el Jira del equipo mediante los caminos establecidos por **Gherkin**.

2.7. Consideraciones de Infraestructura

Para que las pruebas sean repetibles se trabajará con entornos separados y configuración explícita. Se considerará un entorno de desarrollo local para ejecución rápida de pruebas unitarias y de integración, un entorno QA o preproducción para pruebas E2E, UAT y rendimiento. La infraestructura de la AUT incluye una aplicación de administración (monolito NestJS) con su base de datos de administración y un sistema de caché, y una aplicación principal con microservicios NestJS (usuarios y comunidades) con sus bases de datos y cachés correspondientes, además de mensajería por eventos; por ello, para pruebas se requiere disponer de BBDD de pruebas, Redis operativo para caché cuando proceda, y un broker de mensajería disponible.

Las herramientas utilizadas ya han sido comentadas en los apartados anteriores en donde se ha confirmado el uso de: ESLint para revisión de estilo, SonarQube para análisis estático y métricas, Jest para unitarias, Stryker para mutación, jest-dom para pruebas del frontend, supertest y ficheros `.rest` para pruebas de endpoints, Cypress para E2E y JMeter para rendimiento. Se configurará explícitamente el uso de variables de entorno para apuntar a bases de datos de pruebas, y se evitará cualquier configuración que permita por error ejecutar pruebas contra producción, incluyendo validaciones en CI y nombres de entorno diferenciados.

2.8. Suposiciones

Se asume que el alcance del MVP está cerrado para esta iteración y que los endpoints disponibles para CRUD de usuarios, comunidades, causas, acciones y eventos están documentados o al menos estables. Se asume que existe un entorno QA con datos iniciales suficientes para ejecutar UAT y E2E y que es posible resetear el estado de la BBDD de pruebas entre ejecuciones mediante scripts o contenedores efímeros. Se asume que la configuración de autenticación y roles está implementada en la administración y que puede disponerse de credenciales de prueba para perfiles autorizados. Se asume que SonarQube y el pipeline de CI están disponibles para ejecutar análisis y tests de forma automatizada también empleados en la asignatura de Prácticas Continuas.

2.9. Riesgos

A continuación se presenta una matriz de impacto en la Tabla 2.9 donde se valora cada riesgo con **Probabilidad** e **Impacto** en escala 1–5 (1 bajo, 5 muy alto) y se calcula la **Exposición** como $P \times I$. En esta tabla la columna de **Mitigación** aparece al final, como se requiere.

Riesgo	P	I	P×I	Mitigación
Degradación de rendimiento en listados/consultas al crecer los datos	3	4	12	Plan de carga con *JMeter* sobre endpoints críticos, paginación obligatoria, revisión de consultas e índices, y monitorización de tiempos de respuesta en CI/-QA.
Configuración incorrecta de entornos (despliegue/variables mal separadas)	3	4	12	Separación estricta de entornos (test/prod), variables de entorno versionadas por entorno, validaciones en CI antes de desplegar y *checklist* de release.
Disponibilidad limitada del equipo (otras entregas/trabajo externo) que retrase pruebas y correcciones	4	4	16	Planificación realista por hitos, priorización de pruebas críticas, reparto de responsabilidades, y congelar alcance antes de la entrega.
Integración con servicios externos o dependencias inestables (p. ej., API de terceros)	3	4	12	Uso de *mocks/stubs* en tests, entornos sandbox, reintentos controlados, y pruebas de contrato básicas para detectar cambios.
Fugas de defectos por falta de tiempo para ejecutar regresión completa	4	3	12	Mantener una suite mínima automatizada en cada pipeline y ejecutar suites extendidas por módulo cuando haya cambios relevantes.
Pérdida de trazabilidad (no saber qué se ha probado / qué queda pendiente)	3	3	9	Checklist de pruebas por historia de usuario, evidencias de seguimiento en Jira (capturas/logs) y documentación.

2.10. Criterio de finalización

- **Ejecución:** se ejecuta el **100 %** de los casos planificados (o se documenta claramente cualquier exclusión).
- **Resultados:** alta tasa de casos que pasan y **sin defectos bloqueantes** ni de **severidad alta** abiertos.
- **Regresión en CI:** se ejecutan todos los test realizados.
- **Calidad estática (SonarQube):** se cumple el **quality gate** acordado y no hay vulnerabilidades *blocker* abiertas.
- **Linting: ESLint** no reporta errores críticos sin justificar. Se presenta un formateo con **Prettier** uniforme en todos los archivos.

- **Cobertura:** la cobertura de los módulos probados alcanza el **objetivo pactado** para esta iteración que será de al menos un 80 % del código cubierto.
- **Mutación:** *mutation score* objetivo $\geq 0{,}8$ en el módulo seleccionado (descontando mutantes equivalentes). Los mutantes equivalentes son justificados.
- **Rendimiento: APDEX** objetivo $\geq 0{,}8$ con los umbrales definidos; si no se cumple, se justifica y se proponen medidas correctivas antes de cerrar la iteración.
- **Criterio de no-finalización:** cualquier defecto que comprometa **permisos**, **integridad de datos** o la **estabilidad del CRUD** en administración impide cerrar las pruebas, aunque el resto de métricas sean correctas.

3. Resolución de las pruebas

3.1. Pruebas Unitarias

Las pruebas unitarias constituyen el primer nivel de verificación del sistema y tienen como objetivo asegurar el correcto funcionamiento de las unidades mínimas de lógica de negocio de forma aislada. Estas pruebas permiten detectar errores de manera temprana, facilitar la refactorización del código y garantizar que los cambios introducidos no rompen comportamientos previamente validados.

En el contexto de este proyecto, las pruebas unitarias se diseñan siguiendo los principios de aislamiento, determinismo y reproducibilidad, evitando dependencias externas como bases de datos, servicios de terceros o infraestructura. Para ello, se emplean dobles de prueba (mocks) que permiten simular colaboraciones externas y centrarse exclusivamente en la lógica interna de cada componente.

Las pruebas unitarias se estructuran en dos enfoques complementarios: pruebas de caja blanca y pruebas de caja negra. Las primeras se centran en validar la implementación interna del código, cubriendo caminos de ejecución, condiciones y casos límite; las segundas verifican el comportamiento observable de las unidades a partir de sus entradas y salidas, sin considerar su implementación interna. Esta combinación permite alcanzar un mayor nivel de confianza en la calidad y robustez del código base.

Para ejecutar estas pruebas hay que ejecutar el siguiente comando estando dentro de la carpeta `solidarian-id`:

```
npm run test
```

3.1.1. Pruebas de Caja Blanca

Las pruebas de caja blanca realizadas en este proyecto se centran en la validación de la lógica interna del código mediante el análisis de sus estructuras de control. En particular,

se han aplicado pruebas de caminos básicos, apoyadas en el cálculo de la complejidad ciclomática, con el fin de asegurar que todos los caminos independientes de ejecución son cubiertos por los casos de prueba definidos.

Asimismo, se han incorporado pruebas de condición para verificar el correcto comportamiento de expresiones lógicas simples y compuestas, especialmente en validaciones y reglas de negocio. Finalmente, se han aplicado pruebas de bucles para comprobar el funcionamiento correcto de las iteraciones, considerando escenarios de entrada mínima, máxima y casos límite. Estas técnicas permiten aumentar la confianza en la corrección y robustez de la implementación interna.

Utilizaremos la lógica de negocio del inicio de sesión de nuestra aplicación puesto que nos parece un caso crucial sobre el que realizar las pruebas ya que es el punto de entrada a nuestra aplicación principal.

```
async loginUser(
  username: string,
  password: string,
): Promise<Either<Error, boolean>> {
  const usernameOrError = Username.create(username);
  if (usernameOrError.isLeft()) return left(usernameOrError.value);

  const userOrError = await this.usersRepository.findByUsername(
    usernameOrError.value,
  );

  if (userOrError.isLeft()) return left(userOrError.value);

  const passwordMatch = await this.passwordManager.comparePassword(
    password,
    userOrError.value.passwordHash,
  );

  return right(passwordMatch);
}
```

Código 1: Código para el inicio de sesión de usuarios de la aplicación principal

Pruebas de caminos básicos A continuación, se adjunta una de las pruebas de caminos básicos para el inicio de sesión de usuarios.

```
describe('Path 3: User not found (userOrError.value is null)', () => {
  it('should return EntityNotFoundException when user does not exist',
    async () => {
        // Arrange
        const notFoundError = new EntityNotFoundException('User', 'username');
        mockRepo.findByUsername.mockResolvedValueOnce(left(notFoundError));
        // Act
        const result = await service.loginUser(
          VALID_USERNAME,
          VALID_PASSWORD,
        );
        // Assert
        expect(
          result.isLeft() && result.value instanceof EntityNotFoundException,
        ).toBe(true);
        expect(mockPasswordManager.comparePassword).not.toHaveBeenCalled();
    });
});
```

Código 2: Ejemplo de prueba de camino básico

El Listado 2 muestra una prueba unitaria correspondiente a uno de los caminos básicos del proceso de inicio de sesión. En este caso, se simula el escenario en el que el repositorio no encuentra al usuario, devolviendo un error de tipo EntityNotFoundException.

Para ello, se configura el repositorio mediante un mock para que el método findByUsername retorne un resultado erróneo. A continuación, se invoca el método loginUser con credenciales válidas, forzando la ejecución del camino en el que la validación del nombre de usuario es correcta, pero la recuperación del usuario falla.

Las aserciones comprueban que el resultado es un valor de error (Either en estado Left) y que la excepción devuelta es del tipo esperado. Además, se verifica que el método comparePassword no es ejecutado, lo que confirma que el flujo de ejecución se detiene correctamente ante el fallo y no continúa hacia pasos posteriores. Este test valida de forma explícita uno de los caminos independientes identificados en la lógica de negocio.

Pruebas de condición A continuación, se adjunta una de las pruebas de condición para el código relativo al inicio de sesión de usuarios.

```
describe('Condition: usernameOrError.isLeft() = true', () => {
  it('should return error when username contains invalid characters',
    async () => {
      // Arrange
      const invalidUsername = 'user@invalid!';
      // Act
      const result = await service.loginUser(
        invalidUsername,
        VALID_PASSWORD,
      );
      // Assert
      expect(result.isLeft()).toBe(true);
    });
  it('should return error when username exceeds maximum length',
    async () => {
      // Arrange
      const longUsername = 'a'.repeat(51);
      // Act
      const result = await service.loginUser(longUsername, VALID_PASSWORD);
      // Assert
      expect(result.isLeft()).toBe(true);
    });
});
```

Código 3: Ejemplo de prueba de camino básico

Las pruebas mostradas en el Listado 3 corresponden a pruebas de condición sobre el método loginUser. En este caso, se verifican escenarios en los que la validación del nombre de usuario falla, es decir, cuando usernameOrError.isLeft() es true.

El primer test comprueba que se devuelve un error cuando el nombre de usuario contiene caracteres inválidos, mientras que el segundo valida que se gestione correctamente el caso en que el nombre excede la longitud máxima permitida.

Ambas pruebas confirman que el método detecta correctamente entradas no válidas antes de continuar con la lógica de negocio, cumpliendo así con la intención de las pruebas de condición: evaluar cómo se comporta el código ante distintas combinaciones de valores de entrada y asegurar que las decisiones internas se manejan correctamente.

Pruebas de bucles Para la realización de las pruebas de bucles se ha realizado un análisis exhaustivo de la función *sequence* mostrada en el trozo de código 4, donde se cubren todos los caminos posibles de ejecución del bucle *for* presente en la función. Esta función se puede encontrar en la siguiente ruta: `solidarian-id/libs/ddd/src/either.ts`

```
export function sequence<L, R>(eithers?: Either<L, R>[]): Either<L, R[]> {
  const results: R[] = [];
  const eithersArray = Array.isArray(eithers) ? eithers : [];
  for (let i = 0; i < eithersArray.length; i++) {
    const either = eithersArray[i];
    if (either.isLeft()) {
      return left(either.value);
    }
    results.push(either.value);
  }
  return right(results);
}
```

Código 4: Función *sequence*

El fichero de pruebas para la función *sequence* se pueden encontrar en la siguiente ruta: `solidarian-id/libs/ddd/src/test/sequence.spec.ts`.

Se han probado tres condiciones fundamentales de ejecución del bucle: cuando el *array* es indefinido o vacío (el bucle no se ejecuta en absoluto), cuando el bucle se ejecuta una sola vez con un elemento, y cuando se ejecuta múltiples veces. Además, se han incluido pruebas para validar el comportamiento de salida temprana del bucle, específicamente cuando se encuentra un valor *Left* antes de procesar todos los elementos del *array*, demostrando que la función detiene la iteración inmediatamente.

Las pruebas también cubren casos límite críticos: el caso donde se hacen *n-1* iteraciones (cuando el elemento problemático está en la penúltima posición), el caso donde se completan todas las n iteraciones (todos los elementos son *Right*), y un caso donde se documenta que $n+1$ iteraciones son imposibles debido a la condición de límite del bucle i $<$*length*. En conjunto, estas pruebas validan que el bucle se comporta correctamente bajo todas las condiciones posibles de entrada y ejecución, utilizando técnicas de white-box testing para asegurar que cada rama y condición del código se ejecuta correctamente.

En el trozo de código 5 podemos apreciar un ejemplo de prueba de bucle para la función *sequence* vista anteriormente.

Este *test* verifica que dicha función implementa una salida temprana: cuando encuentra un valor *Left* el bucle se detiene inmediatamente en lugar de continuar con los elementos restantes. Esto demuestra que la función no desperdicia recursos procesando elementos que nunca serán utilizados, devolviendo el primer error encontrado tan pronto como lo detecta, lo cual es el comportamiento esperado.

```
// Condition: m passes, m < n (typical number), early exit before finishing
 it('should stop early (m < n passes) when a Left appears before the end', () => {
   // n = 6, m = 3 -> Left at index 2 => loop executes 3 iterations and returns
   const eithers = [
     right<ValidationError, string>('r1'),
     right<ValidationError, string>('r2'),
     left<ValidationError, string>(new ValidationError('Failure at m')),
     right<ValidationError, string>('r4'), // should never be read
     right<ValidationError, string>('r5'), // should never be read
     right<ValidationError, string>('r6'), // should never be read
   ];

   const result = sequence<ValidationError, string>(eithers);

   expect(result.isLeft()).toBe(true);
   if (result.isLeft()) {
     expect(result.value).toBeInstanceOf(ValidationError);
     expect(result.value.message).toBe('Failure at m');
   }
 });
```

Código 5: Ejemplo de prueba de bucle: salida temprana

3.1.2. Pruebas de Caja Negra

Clases de equivalencia

Las pruebas de caja negra del proceso de inicio de sesión se han realizado mediante pruebas *end-to-end* (e2e) contra el endpoint `POST /api/auth/login` expuesto por el controlador `auth-api.controller.ts`. El objetivo es validar el comportamiento *observable* del sistema ante distintas combinaciones de entrada en los campos `email` y `password`, verificando únicamente salidas externas, sin basarse en detalles de implementación interna.

Bajo este enfoque, el resultado del login se puede modelar como dos clases principales:

- **Acceso válido:** credenciales correctas → respuesta `200 OK` (y, en su caso, establecimiento de sesión mediante `Set-Cookie`).
- **Acceso denegado o inválido:** credenciales incorrectas o petición mal formada → respuesta `401 Unauthorized` (fallo de autenticación) o `400 Bad Request` (fallo de validación/DTO).

Aplicando particionado en clases de equivalencia por campo, se han considerado:

Clases de equivalencia para `email`

- **Válido (registrado):** email con formato válido y existente en el sistema.

- **Inválido (no registrado):** email con formato válido pero no existente en el sistema.
- **Inválido (formato):** email con formato incorrecto (p. ej. ausencia de `@` o dominio).
- **Inválido (ausente/vacío):** email no enviado o cadena vacía.

Clases de equivalencia para `password`

- **Válido:** contraseña correcta asociada al usuario.
- **Inválido:** contraseña incorrecta.
- **Inválido (ausente/vacía):** contraseña no enviada o cadena vacía.

Las pruebas e2e cubren las combinaciones más relevantes de estas clases, asegurando que el sistema responde correctamente con `200`, `401` o `400` según corresponda. A continuación se muestran dos ejemplos representativos en las Figuras 6 y 7.

```
it('should login successfully with valid email and password', async () => {
  const response = await request(app.getHttpServer())
    .post('/api/auth/login')
    .send({ email: 'admin@valid.com', password: 'ValidPassword123' })
    .expect(200);

  expect(response.body).toHaveProperty('email', 'admin@valid.com');
  expect(response.headers['set-cookie']).toBeDefined();
});
```

Código 6: Prueba de caja negra: ambos campos válidos

Este primer test valida el caso de *acceso válido*: un usuario previamente existente en la base de datos puede autenticarse con credenciales correctas, obteniendo `200 OK` y el establecimiento de sesión mediante cookie.

```
it('should fail login with valid email and incorrect password', async () => {
  await request(app.getHttpServer())
    .post('/api/auth/login')
    .send({ email: 'admin@valid.com', password: 'WrongPassword' })
    .expect(401);
});
```

Código 7: Prueba de caja negra: email válido, contraseña incorrecta

En este segundo ejemplo se comprueba el caso *email válido + contraseña incorrecta*, que debe producir un `401 Unauthorized`, manteniendo el comportamiento observable esperado ante una autenticación fallida.

Finalmente, cabe destacar que algunas combinaciones (por ejemplo, {email vacío, password válido} o {email válido, password vacío}) pueden no ser introducibles desde la

propia interfaz por validadores del formulario. No obstante, para verificar el comportamiento del sistema ante clientes externos (o peticiones manipuladas), estas combinaciones se prueban directamente contra el endpoint, comprobando que se responde con `400 Bad Request` cuando la petición incumple los requisitos de validación.

Valores límite

Además de las clases de equivalencia, se han aplicado *pruebas de valores límite* (*Boundary Value Analysis*) para comprobar que las validaciones del sistema se comportan correctamente en los extremos de los rangos permitidos. Este tipo de pruebas es especialmente relevante porque los fallos suelen aparecer en condiciones "justo en el borde" (por ejemplo, una longitud mínima aceptada como inválida, o una longitud máxima aceptada por error).

En este caso, el análisis se ha centrado en el *Value Object* `Username`, cuyo comportamiento observable viene determinado por las reglas de validación implementadas: no puede estar vacío, debe respetar una longitud mínima y máxima (`USERNAME_MIN_LENGTH` y `USERNAME_MAX_LENGTH`) y debe cumplir un patrón alfanumérico (`USERNAME_REGEX`). Por tanto, los valores límite naturales del dominio son:

$$L_{min} - 1, \quad L_{min}, \quad L_{min} + 1, \quad L_{max} - 1, \quad L_{max}, \quad L_{max} + 1$$

donde L_{min} y L_{max} representan las longitudes mínima y máxima permitidas.

Las pruebas verifican, desde un punto de vista de caja negra, si la entrada es aceptada o rechazada:

- En los límites **válidos** (L_{min}, $L_{min}+1$, $L_{max}-1$, L_{max}) se espera una creación correcta del `Username` (resultado *Right*).
- En los límites **inválidos** ($L_{min}-1$, $L_{max}+1$) se espera un fallo de validación (resultado *Left*) con el error `InvalidUsernameError`.

A continuación se muestran ejemplos representativos de estas pruebas, donde se generan cadenas con longitudes exactas para ejercitar los límites:

```
it('should create a Username with exactly the minimum allowed length', () => {
  const minUsername = 'a'.repeat(USERNAME_MIN_LENGTH);
  const result = Username.create(minUsername);
  expect(result.isRight()).toBe(true);
});

it('should fail to create a Username with length less than minimum', () => {
  const tooShort = 'a'.repeat(USERNAME_MIN_LENGTH - 1);
  const result = Username.create(tooShort);
  expect(result.isLeft()).toBe(true);
});
```

Código 8: Valores límite en `Username`: L_{min} y $L_{min} - 1$

En el primer caso se prueba el límite inferior exacto (L_{min}), que debe ser aceptado. En el segundo caso se prueba el valor inmediatamente por debajo ($L_{min} - 1$), que debe rechazarse. De forma análoga, se evalúan el máximo permitido y su vecino inmediato:

```
it('should create a Username with exactly the maximum allowed length', () => {
  const maxUsername = 'a'.repeat(USERNAME_MAX_LENGTH);
  const result = Username.create(maxUsername);
  expect(result.isRight()).toBe(true);
});

it('should fail to create a Username with length greater than maximum', () => {
  const tooLong = 'a'.repeat(USERNAME_MAX_LENGTH + 1);
  const result = Username.create(tooLong);
  expect(result.isLeft()).toBe(true);
});
```

Código 9: Valores límite en `Username`: L_{max} y $L_{max} + 1$

3.2. Pruebas de Análisis Estático

Con el objetivo de reforzar la calidad del código y evitar la introducción de regresiones en la rama `develop`, se integró SonarQube dentro del *pipeline* de Integración Continua (CI) para la asignatura de Practicas Continuas. De esta forma, cada *push* o *pull request* ejecuta el análisis estático del repositorio y, en caso de no superar el *Quality Gate*, el flujo de CI falla, impidiendo que la *pull request* pueda considerarse apta para ser integrada. Este enfoque permite *"echar atrás"* cambios que degradan el código antes de que lleguen a la rama principal de desarrollo.

Integración en el *workflow* de GitHub Actions El *workflow* levanta una instancia de SonarQube en el propio runner mediante `services` de GitHub Actions, junto a una base de datos PostgreSQL, y posteriormente ejecuta el escaneo sobre el código TypeScript. Además, se configura el escaneo para esperar de forma síncrona a la evaluación del *Quality Gate*, de modo que el *job* falle si no se cumple el umbral de calidad. En la Figura 10 se muestra un extracto de la definición de servicios, y en la Figura 11 el parámetro clave que fuerza el bloqueo por *Quality Gate*.

```yaml
services:
  db:
    image: postgres:17-alpine
    env:
      POSTGRES_USER: sonar
      POSTGRES_PASSWORD: sonar
      POSTGRES_DB: sonar
    options: >-
      --health-cmd="pg_isready -U sonar -d sonar"
      --health-interval=10s
      --health-timeout=5s
      --health-retries=30

  sonarqube:
    image: sonarqube:25.12-community
    env:
      SONAR_JDBC_URL: jdbc:postgresql://db:5432/sonar
      SONAR_JDBC_USERNAME: sonar
      SONAR_JDBC_PASSWORD: sonar
    ports:
      - 9000:9000
```

Código 10: Servicios de SonarQube y PostgreSQL levantados en el runner de CI

```yaml
- name: Run SonarQube Scanner
  uses: SonarSource/sonarqube-scan-action@v7
  env:
    SONAR_TOKEN: ${{ env.SONAR_TOKEN }}
    SONAR_HOST_URL: http://localhost:9000
  with:
    args: >
      -Dsonar.sources=apps
      -Dsonar.inclusions=apps/**/src/**/*.ts
      -Dsonar.exclusions=**/dist/**,**/node_modules/**,**/coverage/**,
       **/*.spec.ts,**/*.test.ts,**/*.e2e-spec.ts,**/*.js,
       **/*.json,**/*.css,**/*.html
      -Dsonar.javascript.lcov.reportPaths=coverage/lcov.info
      -Dsonar.coverage.exclusions=**/*.entity.ts,**/*.dto.ts,**/*.filter.ts,
      **/*.guard.ts,**/*.mapper.ts,**/*.port.ts,**/fake*.ts,**/enums/**,
      **/model/**,**/models/**,**/main.ts,**/*.module.ts,**/jest*.config.ts,
      **/*.config.ts,**/*.d.ts,**/index.ts
      -Dsonar.qualitygate.wait=true
```

Código 11: Ejecución del escáner y espera del *Quality Gate* (`sonar.qualitygate.wait=true`)

Aspectos clave del análisis De cara a mantener el análisis alineado con el proyecto, se configuraron los directorios de código fuente, las inclusiones/exclusiones y la ruta de cobertura en formato `lcov`. En particular, se analiza el código TypeScript bajo

`apps/**/src`, se excluyen artefactos generados y dependencias (`dist`, `node_modules`, `coverage`), y se importan métricas de cobertura para que SonarQube pueda calcular la cobertura de las líneas ejecutadas durante las pruebas unitarias como de e2e/integración.

Para que la métrica de *coverage* refleje de forma más fiel el valor aportado por las pruebas, se definieron exclusiones específicas en `sonar.coverage.exclusions`. El criterio seguido fue evitar penalizar el porcentaje de cobertura con ficheros *principalmente declarativos o estructurales*, cuyo comportamiento es trivial, repetitivo o dependiente del framework y no aporta el mismo retorno que cubrir lógica de negocio. En concreto, se excluyeron: (i) clases de persistencia y transporte como `*.entity.ts` y `*.dto.ts`, que suelen ser contenedores de propiedades/anotaciones (ORM/validadores) con poca o nula lógica; (ii) elementos de infraestructura y pegamento (`*.module.ts`, `main.ts`, `index.ts`), centrados en dependencias y arranque; (iii) artefactos de configuración (`jest*.config.ts`, `*.config.ts`, `*.d.ts`), que no representan lógica ejecutable de dominio; (iv) capas de adaptación o mapeo (`*.mapper.ts`, `*.port.ts`) y utilidades auxiliares (`fake*.ts`), cuyo testing directo suele ser redundante si la lógica principal queda validada a través de casos e2e/integración; y (v) directorios de modelos/enums (`**/model/**`, `**/models/**`, `**/enums/**`) cuando actúan como definiciones de tipos/constantes. Con ello, la cobertura se centra en la lógica relevante (servicios, casos de uso, validaciones no triviales, controladores y flujos principales), y se evita que el indicador se distorsione por archivos generados, estructurales o de bajo riesgo.

Quality Gate aplicado Se utilizó el *Quality Gate* por defecto proporcionado por SonarQube, denominado *Sonar way*, ya que establece umbrales razonables para un desarrollo iterativo y evita que se introduzcan problemas en *código nuevo* (enfoque *Clean as You Code*). La Tabla 2 resume las condiciones principales aplicadas sobre el código nuevo.

Métrica (código nuevo)	**Operador**	**Umbral**	**Interpretación**
Nuevas violaciones (`new_violations`)	$>$	0	No se permiten nuevas incidencias en el código nuevo.
Cobertura nueva (`new_coverage`)	$<$	80 %	Se exige al menos un 80 % de cobertura en el código nuevo.
Duplicación nueva (`new_duplicated_lines_density`)	$>$	3 %	Se limita la duplicación del código nuevo a un máximo del 3 %.
Hotspots revisados (`new_security_hotspots_reviewed`)	$<$	100 %	Deben revisarse el 100 % de los *security hotspots* del código nuevo.

Cuadro 2: Condiciones del *Quality Gate* por defecto (*Sonar way*) para *Clean as You Code*.

Este conjunto de condiciones nos pareció adecuado para el proyecto, ya que incentiva buenas prácticas sin bloquear por deuda histórica, focalizándose en evitar que el código nuevo reduzca la mantenibilidad (nuevas incidencias y duplicación), la fiabilidad (problemas detectados) o la seguridad (revisión de *hotspots*), y asegurando un mínimo de cobertura en los cambios introducidos. Además en el CI se nos permite descargar el artefacto y nos especifica en el reporte que zonas y lineas tienen complejidades cognitivas

excesívamente altas que se deben corregir. Todo este coverage visto localizado en la carpeta *coverage/lcov.info* se monta a partir de un comando *npm run test:all:coverage* que concatena tanto todos los test unitarios junto con las pruebas end-to-end.

Resultados del reporte de SonarQube Tras cada ejecución del CI, SonarQube ofrece un reporte mediante el plugin CNES, así como métricas de cobertura y duplicación como se mencionaron anteriormente. En el apartado siguiente se mostrarán imágenes de dicho reporte.

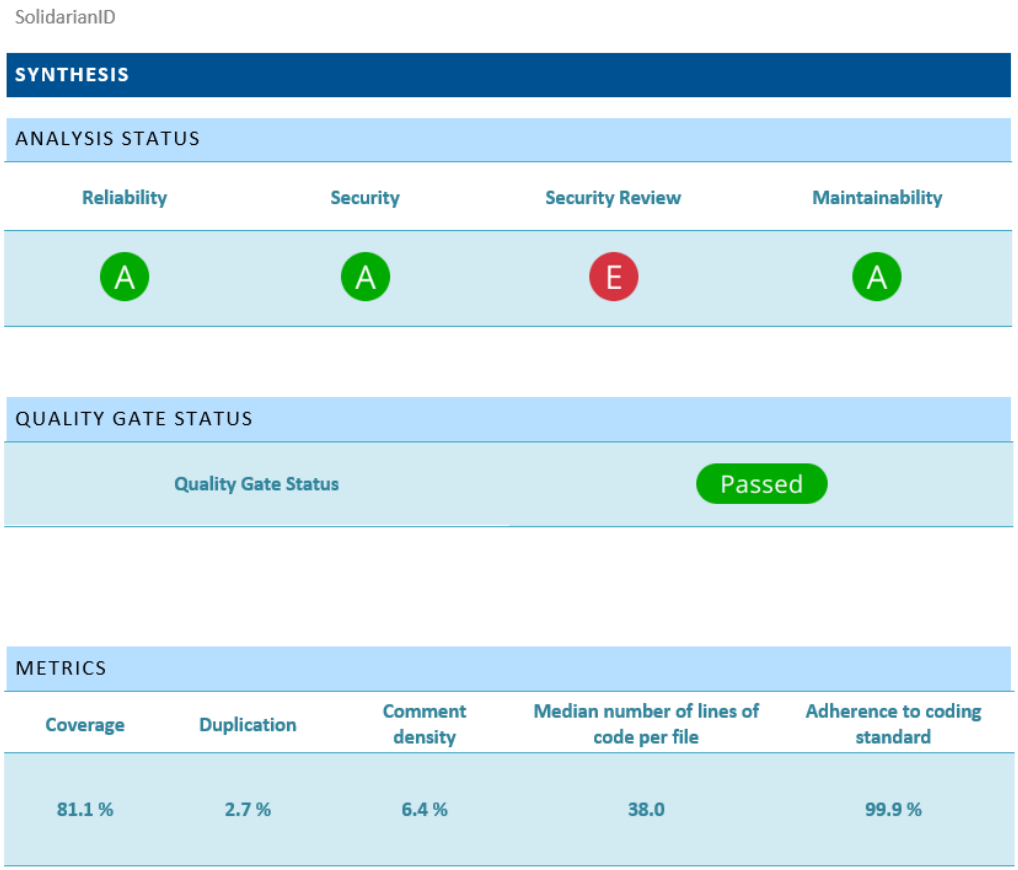


Figura 1: Resumen del reporte de SonarQube para *SolidarianID*: estado del análisis, *Quality Gate* y métricas principales.

En la figura 1 se observa que, dentro de *Analysis Status*, se alcanzan 3 de 4 estándares con calificación **A** (Reliability, Security y Maintainability), mientras que **Security Review** aparece con calificación **E**. Esta desviación suele estar relacionada con la existencia de *security hotspots* pendientes de revisión o con hallazgos potenciales de exposición de secretos (por ejemplo, la presencia de variables en nuestro `.env.example` o valores/constantes en el propio código que deberían omitirse o enmascararse). Adicionalmente, en esta fase del proyecto no se han incorporado mecanismos de endurecimiento típicos en aplicaciones HTTP como *rate limiting* y *Helmet*: el primero limita el número de peticiones por ventana de tiempo (p. ej., por IP o token) para mitigar abusos, fuerza bruta y DoS que se tratarán en las pruebas de rendimiento y carga; el segundo aplica cabeceras HTTP de seguridad (p. ej., para reducir riesgos de *clickjacking*, *MIME sniffing* o políticas de

contenido). A pesar de ello, el *Quality Gate* se muestra como **Passed**, lo que indica que las condiciones configuradas para el código evaluado (especialmente sobre *código nuevo* bajo la filosofía *Clean as You Code*) se han cumplido.

En cuanto a métricas, el reporte refleja una **cobertura** del **81.1 %**, que supera el umbral habitual del 80 % exigido en el *Quality Gate*. Este valor es susceptible de incrementarse en los próximos días, ya que se continúa ampliando la batería de pruebas e incorporando nuevos casos conforme evoluciona la implementación. Asimismo, la **duplicación** se sitúa en **2.7 %**, manteniéndose dentro de valores aceptables. Por último, se destaca que parte de la *cognitive complexity* puede verse penalizada por el patrón *Either* empleado para el manejo explícito de errores (encadenamiento de comprobaciones y ramificaciones), una consecuencia difícil de eliminar sin renunciar a dicho estilo de control de flujo; en estos casos, se asume como un compromiso razonable a cambio de mayor robustez y trazabilidad de errores.

3.3. Pruebas de Mutación

Vamos a realizar pruebas de mutación con Stryker para el microservicio de usuarios de nuestro SolidarianID. Dentro de este, la clase sobre la que hemos realizado tests unitarios es UsersServiceImplementation, es decir, la implementación del servicio de la capa de aplicación que contiene la lógica de negocio para la coordinación entre modelo e infraestructura para el registro, *login*, *logout* y modificación de datos de usuarios registrados en la plataforma. Para ejecutar `Stryker`, usamos el fichero de configuración `stryker.config.json` que podemos ver en el Código 12. Como podemos ver en el fichero, estamos indicando a Stryker que, usando Jest como motor de testing y con cobertura *per-test*, nos mute el servicio de usuarios, así como la clase User (la entidad del modelo del dominio que define al usuario) y todos los value objects que tiene asociados. Además, le indicamos unos límites basados en la cobertura que vamos a ver ahora después y le indicamos que excluya la búsqueda de carpetas a las que no tiene acceso y/o no debe acceder.

```
  {
  "$schema": "./node_modules/@stryker-mutator/core/schema/stryker-schema.json",
  "_comment": "This config was generated using 'stryker init'...",
  "packageManager": "npm",
  "reporters": ["html", "clear-text", "progress"],
  "testRunner": "jest",
  "testRunner_comment": "Take a look at...",
  "coverageAnalysis": "perTest",
  "tempDirName": ".stryker-tmp",
  "cleanTempDir": true,
  "symlinkNodeModules": false,
  "jest": {
    "configFile": "jest.config.ts",
    "config": {
      "testPathIgnorePatterns": [
        "/node_modules/",
        "/apps/admin/",
        "/apps/communities/"]
    }
  },
  "thresholds": {
    "high": 70,
    "low": 60,
    "break": 50
  },
  "mutator": {
    "excludedMutations": ["StringLiteral"]
  },
  "timeoutMS": 60000,
  "maxConcurrentTestRunners": 2,
  "incrementalFile": ".stryker-tmp/incremental.json",
  "mutate": [
    "apps/users/src/application/services/users.service.impl.ts",
    "apps/users/src/domain/model/user.ts",
    "libs/value-objects/src/model/username.ts",
    "libs/value-objects/src/model/mobile-phone.ts",
    "libs/value-objects/src/model/hashed-password.ts",
    "libs/value-objects/src/model/fullname.ts",
    "libs/value-objects/src/model/user-location.ts",
    "libs/value-objects/src/model/city.ts",
    "libs/value-objects/src/model/country-code.ts",
    "libs/value-objects/src/model/unique-entity-id.ts",
    "!**/*.spec.ts",
    "!**/*.e2e-spec.ts",
    "!**/test/**",
    "!**/main.ts"
  ],
  "ignorePatterns": [
    "data", "data/**/*", "dist", "node_modules", ".git", "coverage", "reports"]
}

  }
```


Código 12: Fichero de configuración de Stryker

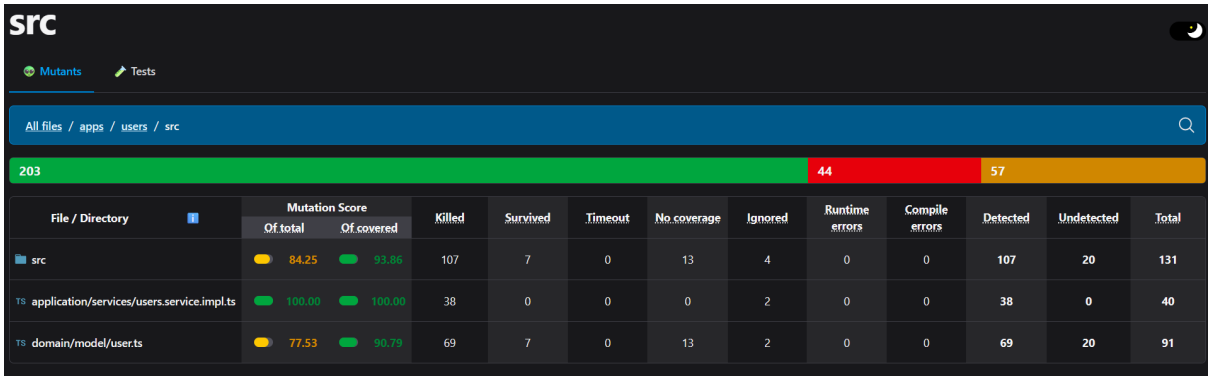


Figura 2: Resultados de testing de mutaciones en usuarios

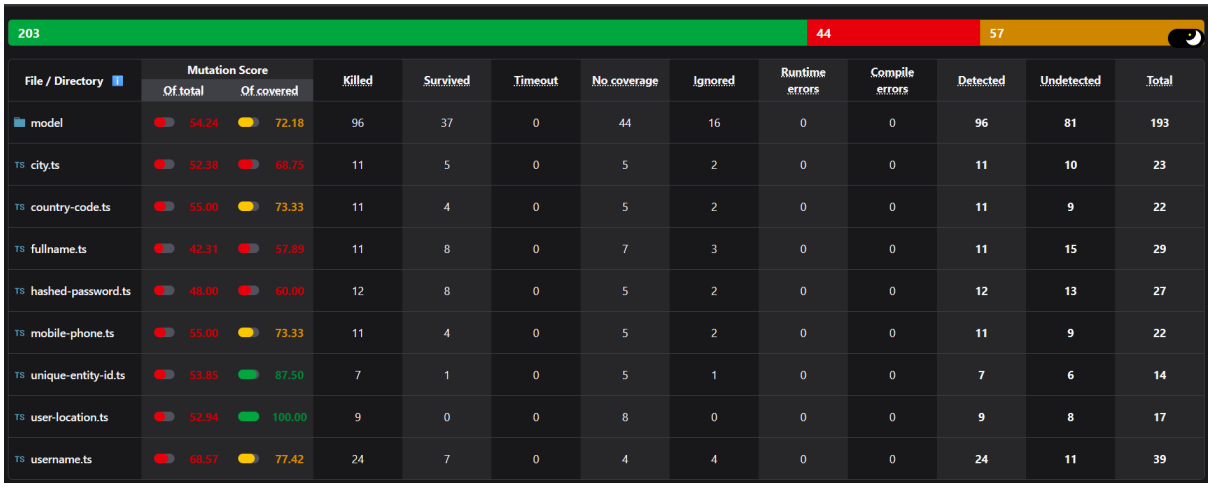


Figura 3: Resultados de testing de mutaciones en Value Objects

Una vez revisada la configuración de Stryker pasamos a ver la cobertura que nos proporciona para hacernos una idea de la calidad de nuestros tests. En las figuras 2 y 3 podemos ver los resultados del *report* generado por Stryker al ejecutarse sobre nuestro código.

En la capa de aplicación, quizás la más crítica, pues es donde reside la lógica de negocio, tenemos un 100 % de *mutation score*, lo que quiere decir que se han eliminado todos los mutantes y que el código está cubierto.

En cuanto al dominio, tenemos más de un 75 %. En total, se han quedado vivos 20 mutantes, 13 de los cuales son por no tener cobertura. Es normal que, en nuestra situación, no se llegue a cubrir el dominio. Hemos preparado unas entidades del dominio que contienen la lógica necesaria para implementar el MVP, pero en la convocatoria en la que presentamos este entregable es imposible haberlo desarrollado entero (y tampoco se espera que lo hagamos). Los siete mutantes vivos están en condiciones específicas como actualizar el país de un usuario en su localización (solo hemos cubierto el cambio de ciudad) o que el fullName se actualice con exactamente 2 caracteres (el mínimo).

En cuanto a los value objects, nos enfrentamos a la misma problemática que con el dominio. En este caso, mucho más acuciante. 44 mutantes sin cobertura (la gran mayoría de los 57 totales) provienen de los métodos *equals* de los VOs, los cuales no hemos cubierto

porque no se contemplan pruebas unitarias de estas clases, siguiendo el principio de testing en pirámide invertida, con muchos tests de integración y menos específicos clase por clase.

Subir a un 85-90 % de puntuación nos supondría más de 100 tests adicionales, cubriendo casos como "username con solo espacios" tan específicos que supondrían un tiempo de desarrollo considerable para el alcance de la aplicación y el tiempo de entrega. Por ello, creemos que el 66,78 % (que en realidad es más alto si eliminamos el "código muerto" de los *equals*) cumple con una cobertura de mutantes suficiente.

3.4. Pruebas de Front-End

Para la implementación de las pruebas *frontend* se ha seleccionado el formulario de inicio de sesión como caso de estudio, debido a su importante funcionalidad dentro del sistema. Para asegurar su correcto funcionamiento y cumplimiento de estándares, se han desarrollado las pruebas utilizando *Jest* y la librería `@testing-library/jest-dom`, que permite simular y validar la estructura del *DOM* en un entorno aislado (*jsdom*).

El fichero de pruebas *frontend* se puede encontrar en la siguiente ruta:

`solidarian-id/apps/admin/test/unit/auth/pages/login.spec.ts`.

El primer paso para realizar las pruebas es configurar el *DOM* para *Jest* en cada test. Para ello configuramos el *beforeEach*, tal como se puede apreciar en el trozo de código 13. Se realizan los siguientes pasos:

- Lee el archivo HTML/Handlebars (`login.hbs`).
- Carga su contenido en el *DOM* virtual mediante `document.body.innerHTML = hbsContent`

A continuación se muestra el trozo de código 13 en cuestión:

```
beforeEach(() => {
    const loginHbsPath = path.resolve(
      process.cwd(),
      'apps/admin/views/pages/login.hbs'
    );
    const hbsContent = fs.readFileSync(loginHbsPath, 'utf8');
    document.body.innerHTML = hbsContent;
});
```

Código 13: *beforeEach* para las pruebas *frontend*

Las pruebas se estructuran en varios bloques que cubren aspectos fundamentales. Cada suite describe un conjunto de expectativas claras y reproducibles. A continuación se muestran algunos ejemplos dentro de cada suite:

- **Estructura del formulario**: se valida la correcta renderización de los elementos

básicos, como se demuestra en la prueba del campo de *email*. En este ejemplo se verifica que exista un input con ID *email*, tipo *email*, atributo *required* y un patrón de validación que incluya el símbolo @. Esta comprobación asegura que la base del formulario cumple con los estándares *HTML* y está preparada para la validación nativa del navegador.

- **Atributos de validación**: se examinan atributos específicos de validación y accesibilidad. El ejemplo que destaca es la prueba que confirma que el campo de contraseña tiene los atributos *minlength=12* y *pattern*, además de *aria-required=true*. Esto es fundamental para garantizar que la contraseña cumpla con políticas de seguridad y que los usuarios de lectores de pantalla sean notificados sobre su obligatoriedad.

- **Interacciones del usuario**: se simula el comportamiento del usuario, como en la prueba del botón de visibilidad de contraseña, donde se cambia dinámicamente el tipo del input entre *password* y *text*. Esta interacción es clave para la experiencia de usuario, permitiendo verificar la contraseña introducida sin comprometer la seguridad base del campo.

- **Estilos y clases *CSS***: se verifica la aplicación correcta de clases *CSS*, como por ejemplo la presencia del contenedor *.auth-form* en el formulario, lo que asegura que los estilos visuales se apliquen consistentemente y mantengan la coherencia con el diseño del sistema.

- **Accesibilidad**: se prioriza la inclusividad, verificando por ejemplo que el botón de alternar visibilidad de contraseña tenga un *aria-label* descriptivo *Mostrar contraseña* y el atributo *aria-controls* asociado al ID del campo. Esto asegura que usuarios con tecnologías asistivas puedan entender y operar esta funcionalidad correctamente.

- **Estados de error**: se centra en la gestión de errores, comprobando que los mensajes de error estén inicialmente vacíos y que puedan actualizarse dinámicamente, manteniendo su asociación con los campos mediante *aria-describedby*. Esto es esencial para una retroalimentación clara y accesible durante la validación del formulario.

Estas pruebas *frontend* demuestran la robustez del componente de *login*. Al integrarse en un *pipeline* de CI/CD, garantizan que cualquier cambio futuro no rompa la funcionalidad crítica, manteniendo la calidad en el sistema.

3.5. Pruebas de Back-End

Para realizar las pruebas *backend* y testear los diferentes *endpoints* de la aplicación se han utilizado diferentes técnicas, como son los ficheros *.rest* y las llamadas a los *endpoints* definiendo asertos con *supertest*. Además, para las pruebas se ha utilizado la tecnología *test container*, la cual permite levantar y eliminar contenedores *Docker* de forma programática. Esto resulta tremendamente útil a la hora de levantar una base de datos de prueba durante la ejecución de los test y eliminarla una vez se ejecutan todos. Para ello se han configurado las variables de entorno en el fichero `.env.test` con el fin de realizar la conexión. En el trozo de código 14 podemos apreciar los 3 métodos `beforeAll(), beforeEach(), afterAll()`.

Estas pruebas se pueden encontrar en los ficheros con extensión `e2e-spec.ts`. En concreto, el fichero del siguiente ejemplo pertenece al fichero ubicado en la ruta:

`solidarian-id/apps/admin/test/administrator/requests.e2e-spec.ts`.

```
    beforeAll(async () => {
        const databaseModule: DynamicModule = await MongoDBTestModule.forRoot();
        server = new TestServerHelper();
        app = await server.create({
          imports: [databaseModule, RequestsModule],
        });

        const connection: Connection = app.get<Connection>('Connection');
        seeder = new MongoDBSeeder(connection);
        repository = app.get<RequestsRepositoryPort>(RequestsRepositoryPort);
    });

    beforeEach(async () => {
        await seeder.clean();
        await seeder.seedRequests();
    });

    afterAll(async () => {
        await server.close();
    });
```

Código 14: Configuración inicial de *test containers* para levantar y eliminar *BD*

A continuación se explica el funcionamiento de cada método:

- `beforeAll()`: se ejecuta una única vez al inicio para inicializar el servidor (`server`, al que se harán las peticiones `HTTP`), y el repositorio de solicitudes (`repository`, el cual servirá como BD de pruebas).
- `beforeEach()` se ejecuta antes de cada test para limpiar la BD (`seeder.clean()`) y recargar los datos de prueba iniciales (`seeder.seedRequests()`), asegurando que cada test tenga un estado limpio e independiente.
- `afterAll()`: se ejecuta una sola vez al final para cerrar el servidor y liberar recursos (`server.close()`).

Una vez detallados los pasos a seguir antes y después de los *test*, pasaremos a explicar el ejemplo mostrado en el trozo de código 15. Como podemos observar, dicho *test* pertenece a la *suite* de pruebas que abarca la actualización (`PATCH`) de solicitudes de creación por parte del administrador, lo que se traduce en validar o rechazar las solicitudes que le lleguen.

En este *test* se manda una petición `PATCH` al *server* con el estado `APPROVED` para aprobar la solicitud, y después se consulta el repositorio para comprobar que efectivamente esa

solicitud ha cambiado al estado `APPROVED`.

```
describe('PATCH /requests/:id', () => {
it('should approve a pending request stored in database', async () => {
  await server
    .getRequest()
    .patch(`/requests/${pendingRequestId}`)
    .send({
      status: RequestStatus.APPROVED,
      reason: 'Database test approval',
    })
    .expect(204);

  const result = await repository.findById(pendingRequestId);
  expect(result.isRight()).toBe(true);
  if (result.isRight()) {
    expect(result.value.status).toBe(RequestStatus.APPROVED);
  }
});
```

Código 15: Ejemplo pruebas *backend* con *test containers*

A continuación mostraremos un ejemplo de pruebas con ficheros *.rest*, tal como se puede observar en el trozo de código 16. En este ejemplo, se prueba también el *endpoint* para validar una solicitud de creación por parte del administrador.

```
@baseUrl = http://localhost:3000
@requestsEndpoint = {{baseUrl}}/requests
@contentType = application/json
@request-id = "518ca02b-7db9-492e-97a4-62ccb20a7dad"
@non-request-id = "0b5320ee-07e5-4974-8756-fb58651a6535"
@{{@already-approved-request-id}} = "d4f5e6c7-89ab-4cde-f012-3456789abcde"

### 1. Test approving a pending request with an optional reason
PATCH {{requestsEndpoint}}/{{request-id}}
Content-Type: {{contentType}}
{
  "status": "approved",
  "reason": "All requirements have been met. Documentation is complete."
}
```

Código 16: Ejemplo pruebas *backend* con *.rest*

Este fichero de pruebas *.rest* para los *endpoints* de las solicitudes del administrador se puede encontrar en la siguiente ruta:

`ccps/request/admin/requests.rest`.

3.6. Pruebas de End-To-End

Las pruebas *End-To-End* (E2E) verifican el comportamiento del sistema desde la perspectiva del usuario, ejecutando la aplicación en un navegador y comprobando que los flujos principales funcionan correctamente de principio a fin (UI → llamadas HTTP → cambios observables). En SolidarianID se han implementado estas pruebas con **Cypress**, aprovechando su ejecución en navegador real, sus esperas automáticas y su facilidad de depuración para detectar regresiones en flujos críticos del panel de administración.

3.6.1. Indicaciones para ejecutar estas pruebas (para la corrección)

Las indicaciones para la ejecución de estas pruebas se encuentran en la raíz del repositorio en el siguiente fichero:

`ccps/cypress/README.md`

3.6.2. Reutilización y mantenibilidad

Para mantener los tests concisos, se han definido comandos custom en `support/commands.js`, encapsulando acciones recurrentes del panel y de los microservicios (por ejemplo, autenticación, registro y preparación de datos por API mediante `cy.request`). El fichero `support/e2e.js` centraliza la carga de comandos y mejora la legibilidad de la ejecución reduciendo ruido en el log interno.

Para desacoplar los escenarios de valores fijos y favorecer la repetibilidad, se utilizan *fixtures* JSON (carpeta `fixtures/`). En particular, `admins.json` contiene credenciales válidas y casos inválidos, y `community-requests.json` aporta datos base para solicitudes de comunidad.

3.6.3. Escenarios E2E implementados

Los escenarios se centran en funcionalidades de alto impacto dentro del panel de administración, priorizando flujos completos y representativos.

Especificación	Objetivo y comprobaciones principales
`e2e/auth/login.cy.js`	Acceso de administradores al panel: renderizado del formulario, presencia de elementos clave, autenticación correcta con varios administradores y gestión de errores (credenciales inválidas, campos obligatorios y formato de email). Incluye mostrar/ocultar contraseña.
`e2e/auth/register.cy.js`	Alta de administradores: registro correcto con datos únicos, posibilidad de iniciar sesión tras el registro, control de duplicidad de email y validaciones del cliente (obligatoriedad, formato de email y coincidencia de contraseñas).
`e2e/requests/community-request-flow.cy.js`	Flujo UI+API de solicitudes de comunidad: creación de usuario y solicitud por API, revisión desde el panel y resolución (aprobación o rechazo). Se valida el impacto en la métrica de comunidades del dashboard y se contempla paginación para localizar la solicitud.
`e2e/dashboard/metrics.cy.js`	Métricas del dashboard: verificación del renderizado de los *canvas* de los gráficos (top comunidades, nacionalidades, ODS y altas mensuales) y comprobación de que cada gráfica contiene datos reales renderizados. Para ello, además de validar visibilidad y existencia de los elementos, se inspeccionan píxeles del *canvas* (ImageData) para asegurar que no se trata de un render vacío.

Cuadro 3: Escenarios E2E de Cypress implementados en el panel de administración.

3.6.4. Preparación de estado y estabilidad de las pruebas

Para asegurar independencia entre casos, los tests limpian el estado del navegador (cookies y *localStorage*) antes de ejecutarse. En flujos que lo requieren, se prepara el estado autenticado de forma eficiente mediante petición HTTP al endpoint de login del panel, manteniendo la intención E2E pero minimizando tiempo y puntos de fallo.

Además, se generan identificadores únicos cuando se crean entidades persistentes, lo que mejora la repetibilidad. En el flujo de solicitudes de comunidad se controlan estados intermedios (crear por API, navegar al listado, resolver y verificar desaparición/impacto en métricas).

3.6.5. Ejecución y evidencias

Las pruebas pueden ejecutarse en modo interactivo (depuración) o en modo headless (CI). Los resultados y evidencias generadas (vídeos y, cuando aplica, capturas) se almacenan en la carpeta de Cypress dentro del repositorio: `solidarian-id/apps/admin/test/cypress/`. Esto permite al profesorado revisar la ejecución y el comportamiento observado en cada escenario.

3.7. Pruebas de Rendimiento

Las pruebas de rendimiento tienen como objetivo evaluar el comportamiento del sistema SolidarianID bajo condiciones de carga concurrente, analizando su capacidad de respuesta, estabilidad y tolerancia al error en un entorno lo más cercano posible a producción. Mientras que el resto de pruebas realizadas validan la corrección funcional del sistema, este tipo de pruebas permite verificar que el MVP es capaz de soportar un uso realista sin degradar de forma significativa la experiencia de usuario.

Tal y como se definió en el plan de pruebas (Sección 2.6.6), el foco se ha puesto en los endpoints críticos de la aplicación de administración, empleando métricas objetivas como tiempos de respuesta, throughput, tasa de errores e índice APDEX, con el fin de obtener una visión global del rendimiento del sistema.

3.7.1. Entorno de pruebas y realismo del escenario

Las pruebas de rendimiento se ejecutaron contra un **servidor real desplegado en Amazon Web Services (AWS)**, utilizando una instancia **EC2 t3.medium**. Este enfoque aporta un alto grado de realismo a los resultados obtenidos, ya que introduce limitaciones reales de infraestructura, latencias de red y contención de recursos, que no se dan en ejecuciones puramente locales.

La instancia utilizada presenta las siguientes características:

- **Tipo de instancia**: AWS EC2 t3.medium
- **CPU**: 2 vCPU
- **Memoria**: 4 GB RAM

Las instancias de la familia *t3* están orientadas a cargas de trabajo generales, por lo que los resultados obtenidos pueden considerarse representativos de un escenario real de uso y permiten extraer conclusiones válidas sobre el comportamiento y los límites actuales del sistema.

3.7.2. Herramienta utilizada y diseño del plan de pruebas

Para la ejecución de las pruebas de rendimiento se utilizó **Apache JMeter**, una herramienta ampliamente adoptada para pruebas de carga y estrés en sistemas distribuidos. El escenario de pruebas se definió mediante un fichero de configuración `.jmx`, en el que se modeló un uso típico de la aplicación de administración.

El plan de pruebas incluye los siguientes elementos principales:

- **Thread Group** para simular usuarios concurrentes.
- **Samplers HTTP** representando peticiones reales a la API.

- **Configuración de cabeceras y payloads** equivalente a clientes reales.
- **Listeners** para la recogida de métricas y generación de resultados.

El diseño del test busca reproducir un patrón de carga progresivo, permitiendo observar el comportamiento del sistema tanto en condiciones normales como en escenarios cercanos a la saturación.

3.7.3. Ejecución de las pruebas

Las pruebas de rendimiento se ejecutaron con una duración aproximada de **13 minutos**, alcanzando un volumen total de **104 648 peticiones HTTP**.

La ejecución se realizó de forma continua y sin intervención manual, asegurando que los resultados recogidos reflejan de manera consistente el comportamiento del sistema bajo la carga definida. Durante la prueba no se aplicaron reinicios ni ajustes dinámicos del entorno, con el objetivo de observar la respuesta natural del sistema ante el incremento de demanda.

3.7.4. Generación del reporte de resultados

Una vez finalizada la ejecución, JMeter generó automáticamente un **reporte HTML interactivo** a partir de los ficheros de resultados en formato CSV. Este reporte proporciona una visión agregada y detallada de las métricas recogidas, incluyendo tablas estadísticas, gráficos temporales y resúmenes de errores.

El dashboard principal del reporte incluye información general de la ejecución (fecha de inicio y fin, número total de peticiones), así como accesos a gráficos de throughput, tiempos de respuesta y distribución de errores. Estas visualizaciones facilitan el análisis del rendimiento y la identificación de posibles cuellos de botella.

3.7.5. Resultados obtenidos

A partir del análisis del reporte generado se obtuvieron las siguientes métricas principales:

- **Número total de peticiones**: 104 648
- **Throughput medio**: 136,91 peticiones/segundo
- **Tiempo medio de respuesta**: 183,67 ms
- **Mediana**: 83 ms
- **Percentil 90**: 286 ms
- **Percentil 95**: 1 082 ms

- **Percentil 99**: 1 149 ms
- **Tiempo máximo observado**: 21 048 ms
- **Errores**: 6 328 peticiones (6,05 %)

Estos valores indican que la mayoría de las peticiones se resuelven en tiempos reducidos, mientras que una fracción minoritaria presenta latencias elevadas en escenarios de mayor carga.

3.7.6. Análisis e interpretación de los resultados

La mediana de 83 ms indica que más del 50 % de las peticiones se procesan de forma muy rápida, lo que se traduce en una experiencia fluida para la mayoría de los usuarios. Asimismo, el percentil 90 se mantiene por debajo de los 300 ms, lo que refleja un comportamiento consistente del sistema en condiciones normales de carga.

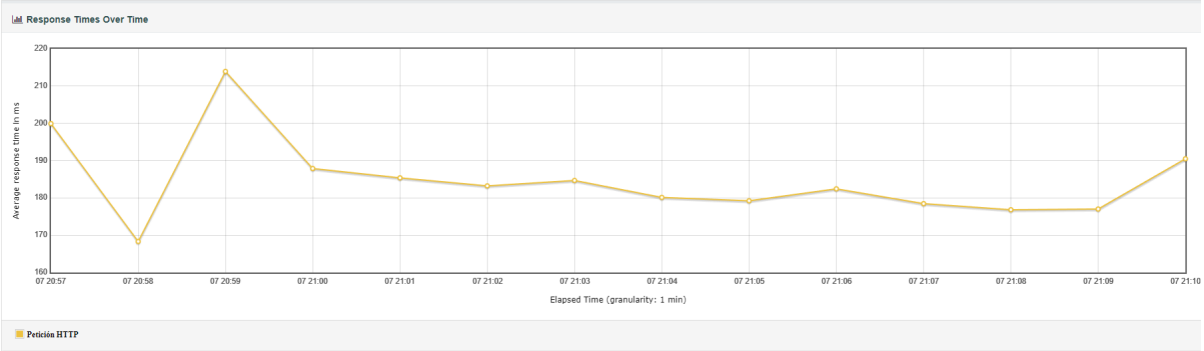


Figura 4: Evolución de los tiempos de respuesta durante la prueba

En la Figura 4 se observa la evolución de los tiempos medios de respuesta a lo largo de la ejecución de la prueba. El gráfico muestra un comportamiento estable durante la mayor parte del tiempo, con ligeras oscilaciones y algunos incrementos puntuales, especialmente en los momentos de mayor concurrencia. No se aprecian crecimientos progresivos ni patrones de degradación sostenida, lo que indica que el sistema mantiene un rendimiento estable durante la prueba.

Los percentiles altos (p95 y p99), junto con el tiempo máximo observado, ponen de manifiesto la existencia de picos de latencia puntuales. Este comportamiento es coherente con la ejecución de pruebas de carga y estrés sobre una instancia *t3.medium*, donde la disponibilidad limitada de recursos puede provocar incrementos temporales en los tiempos de respuesta bajo alta concurrencia.

La tasa de errores observada (6,05 %) se debe mayoritariamente a errores de tipo *Non HTTP response code: java.net.BindException (Address already in use)*. Este tipo de error no corresponde a fallos del sistema bajo prueba, sino a una limitación del propio cliente de carga, relacionada con el agotamiento de recursos locales (puertos efímeros y conexiones TCP) durante la generación intensiva de peticiones. Estos errores se producen cuando

JMeter no puede abrir nuevas conexiones de red a nivel del sistema operativo, antes de que la petición llegue al servidor. Por tanto, no representan errores funcionales ni de estabilidad de la aplicación SolidarianID, sino una señal de saturación del entorno de generación de carga.

Considerando este aspecto, los resultados confirman que el sistema evaluado responde correctamente a la mayoría de las peticiones recibidas y que los errores registrados no comprometen el correcto funcionamiento del backend ni la validez de las pruebas realizadas.

3.8. Pruebas Metamórficas

En este apartado se describen las pruebas metamórficas planteadas para `SolidarianID`. Estas pruebas se diseñan para mitigar el *problema del oráculo* (casos en los que no es trivial conocer el resultado exacto esperado), verificando en su lugar **relaciones necesarias** entre ejecuciones del sistema bajo transformaciones controladas de la entrada o del contexto.

En nuestro caso, las relaciones metamórficas se aplican en dos ámbitos: (i) lógica de negocio del registro de usuarios (servicio de aplicación), y (ii) consistencia de las operaciones de consulta del repositorio de solicitudes, donde la salida se interpreta como *conjuntos* y se validan propiedades de *subconjunto*, *disjunción* y *partición*.

3.8.1. Pruebas metamórficas unitarias

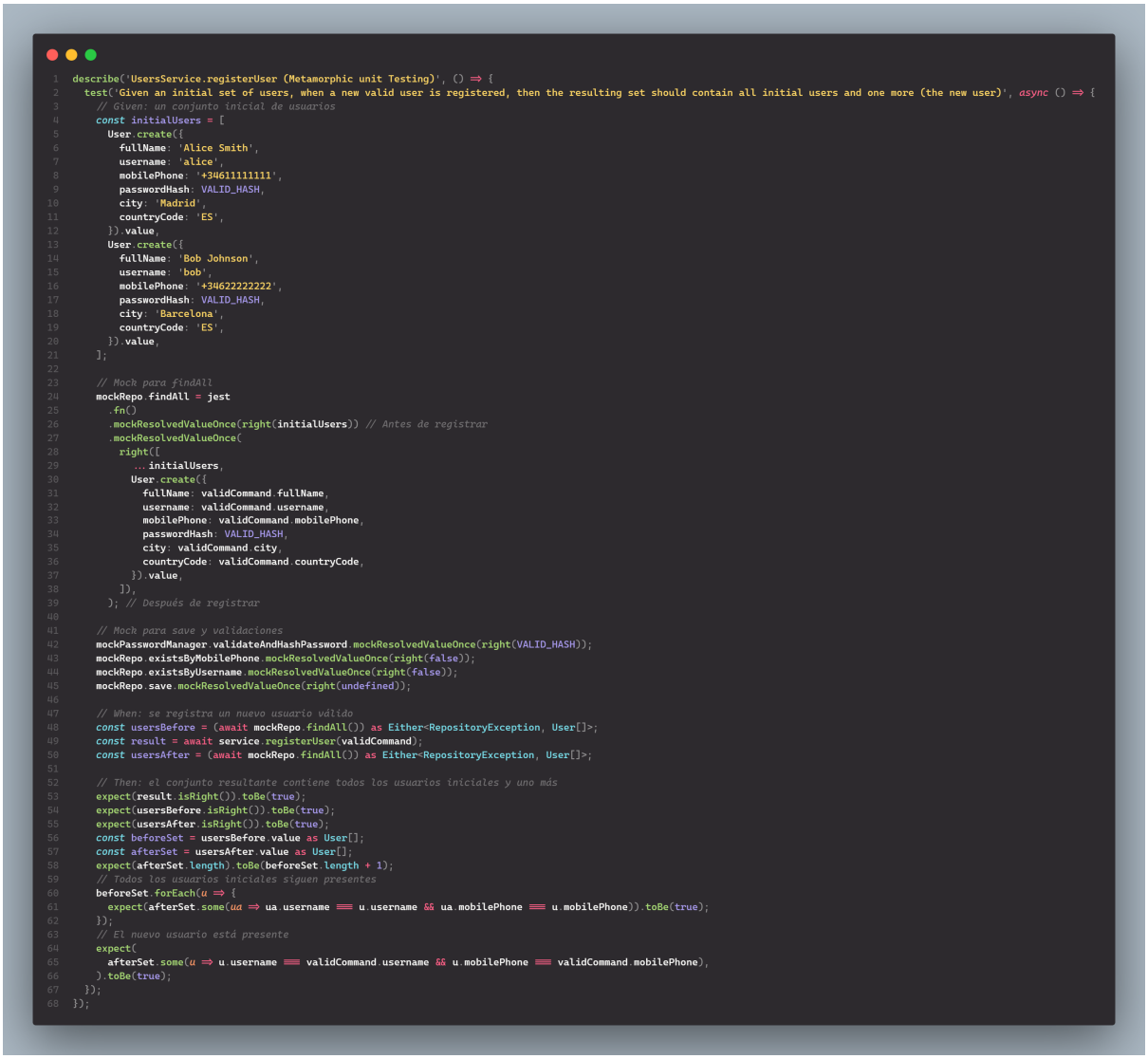
SUT (System Under Test). `UsersService.registerUser`, responsable de registrar un usuario válido y persistirlo si no existen colisiones en credenciales (p. ej., `username` y `mobilePhone`). Se adjunta el código relativo a estas pruebas en la Figura 5

Motivación. En el registro de usuario, el resultado exacto de una operación puede depender de varios factores (validaciones, persistencia, hashing de contraseña, etc.). En lugar de depender de un único oráculo (p. ej., "el repositorio debe contener exactamente X"), se valida una propiedad más robusta: **la colección final de usuarios debe contener a todos los usuarios previos y además incluir el nuevo usuario**.

Relación metamórfica (RM-U1): crecimiento por inserción válida. Sea U_{before} el conjunto de usuarios existente antes del registro y U_{after} el conjunto tras registrar un usuario válido nuevo u_{new}. Entonces debe cumplirse:

$$U_{before} \subseteq U_{after} \quad \wedge \quad |U_{after}| = |U_{before}| + 1 \quad \wedge \quad u_{new} \in U_{after}$$

Caso origen y caso de seguimiento.



```
describe('UsersService.registerUser (Metamorphic unit Testing)', () => {
  test('Given an initial set of users, when a new valid user is registered, then the resulting set should contain all initial users and one more (the new user)', async () => {
    // Given: un conjunto inicial de usuarios
    const initialUsers = [
      User.create({
        fullName: 'Alice Smith',
        username: 'alice',
        mobilePhone: '+34611111111',
        passwordHash: VALID_HASH,
        city: 'Madrid',
        countryCode: 'ES',
      }).value,
      User.create({
        fullName: 'Bob Johnson',
        username: 'bob',
        mobilePhone: '+34622222222',
        passwordHash: VALID_HASH,
        city: 'Barcelona',
        countryCode: 'ES',
      }).value,
    ];

    // Mock para findAll
    mockRepo.findAll = jest
      .fn()
      .mockResolvedValueOnce(right(initialUsers)) // Antes de registrar
      .mockResolvedValueOnce(
        right([
          ...initialUsers,
          User.create({
            fullName: validCommand.fullName,
            username: validCommand.username,
            mobilePhone: validCommand.mobilePhone,
            passwordHash: VALID_HASH,
            city: validCommand.city,
            countryCode: validCommand.countryCode,
          }).value,
        ]),
      ); // Después de registrar

    // Mock para save y validaciones
    mockPasswordManager.validateAndHashPassword.mockResolvedValueOnce(right(VALID_HASH));
    mockRepo.existsByMobilePhone.mockResolvedValueOnce(right(false));
    mockRepo.existsByUsername.mockResolvedValueOnce(right(false));
    mockRepo.save.mockResolvedValueOnce(right(undefined));

    // When: se registra un nuevo usuario válido
    const usersBefore = (await mockRepo.findAll()) as Either<RepositoryException, User[]>;
    const result = await service.registerUser(validCommand);
    const usersAfter = (await mockRepo.findAll()) as Either<RepositoryException, User[]>;

    // Then: el conjunto resultante contiene todos los usuarios iniciales y uno más
    expect(result.isRight()).toBe(true);
    expect(usersBefore.isRight()).toBe(true);
    expect(usersAfter.isRight()).toBe(true);
    const beforeSet = usersBefore.value as User[];
    const afterSet = usersAfter.value as User[];
    expect(afterSet.length).toBe(beforeSet.length + 1);
    // Todos los usuarios iniciales siguen presentes
    beforeSet.forEach(u => {
      expect(afterSet.some(ua => ua.username === u.username && ua.mobilePhone === u.mobilePhone)).toBe(true);
    });
    // El nuevo usuario está presente
    expect(
      afterSet.some(u => u.username === validCommand.username && u.mobilePhone === validCommand.mobilePhone),
    ).toBe(true);
  });
});
```

Figura 5: Pruebas metamórficas unitarias

- **Caso origen:** se parte de un conjunto inicial de usuarios U_{before} (mock del repositorio en la primera llamada a `findAll`).
- **Transformación metamórfica:** se ejecuta `registerUser` con un comando válido `validCommand`.
- **Caso de seguimiento:** se obtiene el conjunto U_{after} (mock del repositorio en la segunda llamada a `findAll`) que debe reflejar el estado posterior.

Oráculo metamórfico (aserciones). Las verificaciones realizadas comprueban: (i) que el resultado de la operación es correcto (`Either.Right`), (ii) que el tamaño del conjunto crece en una unidad, (iii) que todos los usuarios iniciales siguen presentes (inclusión por identidad: `username` y `mobilePhone`), y (iv) que el nuevo usuario aparece en el conjunto final.

Tipo de defectos que detecta. Esta RM permite revelar, por ejemplo:

- fallos de persistencia (no se inserta el usuario),
- fallos de idempotencia/duplicación (se insertan más usuarios de los esperados),
- efectos colaterales no deseados (se pierden usuarios previos),
- errores en reglas de unicidad o validaciones aplicadas incorrectamente.

3.8.2. Pruebas metamórficas e2e

SUT (System Under Test). Operaciones de consulta del `requestsRepository`: `findAll()` y `findByType(type)` sobre solicitudes del sistema. En `SolidarianID` existen tres tipos de solicitudes: `COMMUNITY`, `CAUSE` y `ACTION`. En nuestro código (Fichero *requests.repository.e2e-spec*) se puede observar las pruebas de integración del repositorio de solicitudes con la base de datos haciendo uso de esta técnica.

Estas pruebas se pueden encontrar en los test de integración del repositorio de solicitudes en el microservicio de comunidades.

Motivación. En un repositorio real (con persistencia y datos variables), no siempre interesa fijar un resultado exacto (p. ej., número concreto de filas), sino validar que las consultas mantienen **relaciones lógicas** entre sus resultados. Aquí la salida se interpreta como conjuntos de identificadores.

Relación metamórfica (RM-E1): subconjunto por filtrado. Para cualquier tipo t, si R_t es el conjunto de solicitudes devueltas por `findByType(t)` y R_{all} el conjunto devuelto por `findAll()`, debe cumplirse:

$$R_t \subseteq R_{all}$$

Esta RM se verifica para los tres tipos: `COMMUNITY`, `CAUSE` y `ACTION`. Operacionalmente, se comprueba que cada ID de R_t aparece también en R_{all}.

Relación metamórfica (RM-E2): disjunción por tipos. Dado que una solicitud pertenece exactamente a un tipo, los resultados filtrados por tipos distintos deben ser disjuntos. Para dos tipos distintos $t_1 \neq t_2$:

$$R_{t_1} \cap R_{t_2} = \emptyset$$

En las pruebas se valida por pares: `COMMUNITY` vs `CAUSE`, `COMMUNITY` vs `ACTION` y `CAUSE` vs `ACTION`.

Relación metamórfica (RM-E3): partición completa. La combinación de las tres consultas por tipo debe formar una partición del conjunto total:

$$\begin{aligned} R_{community} \cup R_{cause} \cup R_{action} &= R_{all} \\ R_{community} \cap R_{cause} &= \emptyset \\ R_{community} \cap R_{action} &= \emptyset \\ R_{cause} \cap R_{action} &= \emptyset \end{aligned}$$

En el código, esta RM se materializa comprobando: (i) igualdad de cardinalidad entre la unión de los tres conjuntos y `findAll`, (ii) inclusión bidireccional (todo elemento de la unión está en `all` y viceversa), (iii) intersecciones vacías, y (iv) coherencia entre el tamaño total y la suma de tamaños individuales (propiedad esperable si la partición es correcta).

Caso origen y caso de seguimiento.

- **Caso origen:** R_{all} = `findAll()`.
- **Transformación metamórfica:** aplicar el filtro por tipo t (o por los tres tipos).
- **Casos de seguimiento:** R_t = `findByType(t)` para cada tipo, o bien $\{R_{community}, R_{cause}, R_{action}\}$ para verificar la partición.

Tipo de defectos que detecta. Estas RMs son eficaces para detectar:

- filtrados incorrectos (IDs fuera de su tipo),
- pérdida de elementos (consultas por tipo que omiten solicitudes existentes),
- duplicidades o mezclas de tipo (solicitudes que aparecen en más de un conjunto),
- inconsistencia entre consultas (por ejemplo, `findAll` y `findByType` con criterios divergentes).

3.8.3. Pruebas metamórficas con Katalon

Para realizar las pruebas metamórficas con Katalon en local hemos hecho uso del fichero `docker compose` ubicado en el directorio *deployment* de la raíz del proyecto. Estas pruebas se han hecho accediendo a nuestra aplicación de administración en la interfaz de red virtual (loopback) al puerto 3000.

En concreto, se ha automatizado el flujo de autenticación (*login*) mediante una *test suite* exportada por Katalon Recorder en formato HTML (secuencia de comandos *open/-click/type*). Aunque el fichero contiene tres casos funcionales (*login-test-case-succesful*, *login-test-case-validation-error* y *login-test-case-unauthorized*), su interpretación en este

apartado es metamórfica: no se valida únicamente un resultado aislado, sino relaciones necesarias entre ejecuciones al aplicar transformaciones controladas sobre las entradas del formulario de autenticación. En la Figura 6 se pueden observar cómo las pruebas realizadas pasan correctamente.

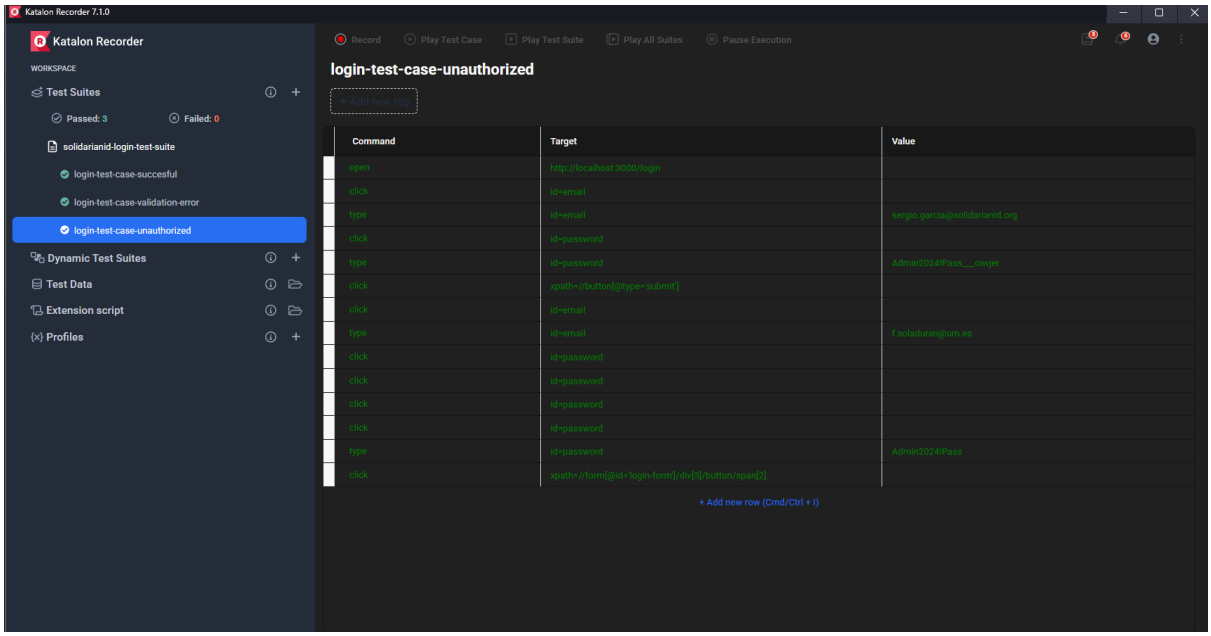


Figura 6: Pruebas metamórficas del login con Katalon

A continuación, explicaremos las relaciones metamórficas que hemos identificado en este sistema bajo prueba.

Sistema bajo prueba (SUT). Pantalla de login accesible en *http://localhost:3000/login* y su comportamiento tras enviar el formulario (`login-form`), incluyendo la transición a `/dashboard` cuando la autenticación es válida.

Representación de resultados. Para formular las relaciones metamórficas, se consideran como observables: (i) la **ruta final** tras enviar el formulario (permanencia en `/login` o navegación a `/dashboard`), y (ii) la presencia de **mensajes de validación** asociados a formato de email (p. ej., "Utiliza un formato que coincida con el solicitado").

Relación metamórfica RM-K1 (baseline de autenticación válida). Se define como caso origen un inicio de sesión con credenciales válidas: *sergio.garcia@solidarianid.org* y *Admin2024!Pass* (caso `login-test-case-succesful`). La relación metamórfica establece que, si las credenciales son correctas, el sistema alcanza un estado autenticado observable como:

$$\texttt{login}(email_{valid}, pass_{valid}) \Rightarrow \texttt{URL} = \texttt{/dashboard}$$

En el fichero de Katalon esto se refleja en el acceso posterior a *http://localhost:3000/dashboard* tras el envío del formulario.

Relación metamórfica RM-K2 (degradación: email inválido ⇒ error de validación / no autenticado). Tomando como referencia el caso origen (RM-K1), se aplica una transformación metamórfica sobre la entrada del formulario que viola una precondición sintáctica: sustituir el email válido por un email inválido (p. ej., *sergio.garcia*) manteniendo el resto del flujo (caso `login-test-case-validation-error`). La relación metamórfica esperada es que el sistema **no autentique** y permanezca en estado de validación:

$$\texttt{login}(email_{invalid}, pass) \Rightarrow \texttt{URL} = \texttt{/login} \ \wedge \ \texttt{validationMessage(email)} \neq \emptyset$$

Esta RM detecta fallos donde el cliente no valida el formato del email o permite el envío sin advertencias.

Relación metamórfica RM-K3 (reparación: corregir el email elimina el error de formato). A partir del estado provocado por RM-K2, se realiza la transformación inversa (*reparación*) corrigiendo el email a un valor bien formado *sergio.garcia@solidarianid.org* y reenviando el formulario (segunda parte del caso `login-test-case-validation-error`). La relación metamórfica no exige necesariamente éxito de autenticación (ya que la contraseña introducida puede no coincidir con la válida), pero sí exige coherencia: el sistema debe **abandonar el estado de error de formato** y pasar a un estado consistente con el backend (éxito si la contraseña es correcta, o no autorizado si no lo es):

$$\texttt{login}(email_{invalid}, pass) \Rightarrow [\text{corrige } email]\texttt{login}(email_{valid}, pass)$$
$$[\text{corrige } email]\texttt{login}(email_{valid}, pass) \Rightarrow \texttt{validationMessage(email)} = \emptyset$$

En términos prácticos, se verifica que el fallo deja de ser "formato inválido" y pasa a depender de la verificación de credenciales.

Relación metamórfica RM-K4 (transformación: contraseña incorrecta ⇒ no autorizado). Partiendo del caso origen (RM-K1), se transforma únicamente la contraseña manteniendo el email válido, introduciendo un valor incorrecto en el caso `login-test-case-unauthorized`. La relación metamórfica establece que el sistema no debe autenticar:

$$\texttt{login}(email_{valid}, pass_{invalid}) \Rightarrow \texttt{URL} = \texttt{/login}$$

Esta RM es útil para detectar defectos de autenticación demasiado permisiva o problemas en el control de sesión (p. ej., sesiones residuales).

Relación metamórfica RM-K5 (independencia del usuario: cambiar email manteniendo password no autentica si no corresponde). En el mismo caso `login-test-case-unauthorized` se realiza un segundo intento modificando el email a *f.soladuran@um.es* y usando *Admin2024!Pass*. La relación metamórfica refuerza que la autenticación depende del par $(email, password)$; por tanto, mantener la contraseña y cambiar el email no debe autenticar si el usuario no existe o no corresponde a dichas credenciales:

$$\texttt{login}(email_1, pass) \neq \texttt{login}(email_2, pass) \quad \text{si} \quad email_1 \neq email_2$$

3.9. Pruebas de Regresión

Al introducir cambios en el código (nuevas funcionalidades, correcciones de errores o mejoras), existe el riesgo de producir *efectos secundarios* no deseados en otras partes del sistema. Las **pruebas de regresión** tienen como objetivo verificar que dichos cambios no introducen nuevos defectos en la funcionalidad ya existente (estabilidad, rendimiento o comportamiento funcional), por lo que deben ejecutarse de forma **periódica** para asegurar que la evolución del software no genera consecuencias imprevistas.

De acuerdo con el enfoque adoptado en esta entrega, se aplicará una estrategia de **pruebas de regresión retest-all (test-all)**: ante cada cambio relevante, se ejecutará la suite completa para comprobar que ninguna funcionalidad previa queda afectada negativamente. Este enfoque se justifica por su **simplicidad operativa** (no requiere análisis adicional de impacto) y por las **limitaciones de tiempo** actuales, asumiendo el mayor coste de ejecución que implica volver a lanzar todas las pruebas. Aun así, se trata de una práctica especialmente recomendable como parte del proceso de entrega, al maximizar la confianza en la estabilidad del sistema tras integrar cambios.

Esta práctica de pruebas de regresión *retest-all* la hemos aplicado en nuestro pipeline de integración continua ya que en uno de sus pasos se ejecuta el siguiente comando:

```
npm run test:all:coverage
```

Dicho *script* está definido en el `package.json` como una **ejecución encadenada** (*fail-fast*) de varias suites de pruebas (unitarias, integración y backend) y, posteriormente, la **generación de informes de cobertura** unificados. En concreto, realiza lo siguiente:

1. **Ejecución de pruebas unitarias:** `npm run test`. Verifica el comportamiento de unidades aisladas (funciones, servicios, componentes) y actúa como primera barrera ante regresiones básicas.

2. **Ejecución de pruebas *end-to-end* (E2E) por servicio:** `npm run test:e2e:admin`, `npm run test:e2e:users` y `npm run test:e2e:communities`. Con ello se valida, de extremo a extremo, el flujo completo de cada módulo/microservicio (rutas HTTP, controladores, lógica y acceso a datos) simulando un uso lo más realista posible.

3. **Ejecución de pruebas de integración:** `npm run test:integration`. Comprue-

ba la interacción entre componentes (p. ej., repositorios, servicios, persistencia y dependencias externas) de manera controlada, detectando fallos que no aparecen en pruebas unitarias.

4. **Recolección de coberturas parciales (Node + NYC):** a continuación se ejecuta un *script* con `node -e ‘‘...‘‘` que recorre varios directorios de cobertura: `coverage`, `coverage-e2e/admin`, `coverage-e2e/users`, `coverage-e2e/communities` y `coverage-e2e/integration`. Para cada uno, si existe el fichero `coverage-final.json`, lo copia a la carpeta `.nyc_output` (creándola si es necesario) con un nombre distinto por suite. De esta forma, NYC puede **agregar** los resultados de cobertura generados por ejecuciones independientes, evitando que se sobrescriban entre sí, consiguiendo la cobertura real de todas nuestras pruebas, tanto unitarias, integración y backend.

5. **Generación de reportes de cobertura agregada:** finalmente se ejecuta `nyc report -reporter=text -reporter=html -reporter=lcov`, produciendo: (i) un resumen en consola (`text`) útil para el *pipeline*, (ii) un informe navegable (`html`) para inspección manual, y (iii) un reporte `lcov` compatible con herramientas de análisis como SonarQube el cual, evidentemente, utilizamos para obtener las métricas de análisis del sonar.

En conjunto, `npm run test:all:coverage` materializa el enfoque *retest-all* al ejecutar de forma consecutiva todas las suites relevantes y, además, proporciona una visión global de la **cobertura** alcanzada por el conjunto de pruebas, lo que facilita detectar áreas del sistema con verificación insuficiente tras cada cambio.

4. Conclusión

A lo largo de este trabajo, la incorporación sistemática de distintas técnicas de prueba ha demostrado ser un elemento clave para garantizar la calidad del desarrollo de *SolidarianID*. Más allá de validar el cumplimiento funcional del *MVP*, el *testing* ha permitido detectar errores de forma temprana, reducir retrabajo y aumentar la confianza en cada iteración del sistema. La combinación de análisis estático con *SonarQube*, pruebas unitarias con *Jest*, mutación con *Stryker*, integración con *TestContainers*, pruebas *end-to-end* con *Cypress*, rendimiento con *JMeter* y pruebas metamórficas con *Katalon* ha proporcionado una cobertura completa desde distintos niveles y perspectivas del sistema.

Un aspecto especialmente relevante ha sido la integración de estas pruebas dentro del *pipeline* de Integración Continua. Este enfoque ha permitido automatizar la verificación de la calidad del código en cada *push* y *pull request*, evitando que errores, regresiones o problemas de calidad lleguen a integrarse en las ramas principales del repositorio. Como resultado, el equipo ha podido trabajar con mayor seguridad, sabiendo que cualquier cambio debía superar una batería de controles automáticos antes de ser aceptado, lo que ha reducido significativamente la probabilidad de introducir fallos en fases avanzadas del desarrollo.

La experiencia práctica ha puesto de manifiesto que el *testing* no es únicamente una fase final de validación, sino una herramienta activa que guía el diseño, mejora la arquitectura

y favorece un desarrollo más sostenible. Herramientas como *ESLint* y *SonarQube* han contribuido a prevenir deuda técnica desde las primeras etapas, las pruebas de mutación han evidenciado debilidades en los casos de prueba, y las pruebas de rendimiento han permitido anticipar problemas de escalabilidad bajo condiciones realistas. Asimismo, las pruebas metamórficas han aportado un valor añadido al validar propiedades esenciales del sistema incluso ante variaciones en los datos de entrada.

En conjunto, este trabajo refuerza la idea de que integrar el *testing* de forma continua no supone un coste adicional, sino una inversión directa en estabilidad, mantenibilidad y profesionalidad del producto. En el contexto de esta asignatura, la experiencia adquirida ha consolidado una visión del *testing* como parte inseparable del ciclo de vida del *software*, y ha dotado al equipo de prácticas y herramientas fundamentales para desarrollar sistemas robustos y fiables en futuros proyectos.

Referencias

[1] Equipo 3. *Repositorio de GitHub del equipo 3*. 2026. URL: https://github.com/fsoladur/SolidarianID.

[2] JEST. *Jest Changed Files*. 2026. URL: https://jestjs.io/docs/jest-platform#jest-changed-files.